

贪心

Bandrupt

西安交通大学

2026 年 7 月 8 日



致谢

感谢校队的同学们，他们在各个方面都帮助了我很多，无论是怎么写 `beamer`，对邻项交换证明的深入讨论，还是挑选和提供例题，我都获得了许多帮助。

也要感谢今天的听众，我第一次做工程量这么大的 PPT，第一次讲课，你们能坐在电脑前认真听我讲课对我而言是莫大的鼓励。如果 PPT 有失误/我讲的不好希望多多包涵，谢谢。

目录

- 1 引入
- 2 邻项交换
- 3 区间选点与区间覆盖
- 4 绝对值模型
- 5 哈夫曼模型
- 6 进阶技巧
- 7 总结

贪心思想

定义： 每一步都选择当前状态下的最优决策，期望通过一系列局部最优解得到全局最优解。

贪心思想

定义： 每一步都选择当前状态下的最优决策，期望通过一系列局部最优解得到全局最优解。

核心问题： 局部最优 $\neq$ 全局最优，需要证明贪心策略的正确性。

基础模型：买最多物品

题目描述

商店里有一些商品，每个商品有一个价格。你的钱数是有限的，如何用这些有限的钱买到尽可能多的商品？（每个商品只能买一次）

基础模型：买最多物品

题目描述

商店里有一些商品，每个商品有一个价格。你的钱数是有限的，如何用这些有限的钱买到尽可能多的商品？（每个商品只能买一次）

贪心直觉：肯定买最便宜的！

基础模型：买最多物品

题目描述

商店里有一些商品，每个商品有一个价格。你的钱数是有限的，如何用这些有限的钱买到尽可能多的商品？（每个商品只能买一次）

贪心直觉：肯定买最便宜的！

策略：每次买剩下的商品中最便宜的商品，直到买不起为止。

基础模型：买最多物品

题目描述

商店里有一些商品，每个商品有一个价格。你的钱数是有限的，如何用这些有限的钱买到尽可能多的商品？（每个商品只能买一次）

贪心直觉：肯定买最便宜的！

策略：每次买剩下的商品中最便宜的商品，直到买不起为止。

将商品的价格从小到大排序，每次把最便宜的商品挑出来，如果钱还够就买，转而考虑下一个商品，否则买不起了。

思维进阶：选物品凑条件

题目描述（由 changtu 同学所出，特别感谢他的贡献！）

游戏里有一些道具。每个道具具有两个属性：攻击力和防御力，第 i 个道具的攻击力记作 a_i ，防御力记作 b_i 。

你可以挑选出一些道具用作攻击，剩下的道具则**全部**用来防御。要求挑选结束后，攻击道具的攻击力总和要大于防御道具的防御力总和。最少挑出多少个道具才能满足这个条件？

例如，总共三个道具，攻防分别是 $(1, 5)(2, 3)(3, 1)$ ，其中第一项是攻，第二项是防。只选第一个不行，这时攻击力总和 $= 1$ ，防御力总和 $= 3 + 1 = 4$ ， $1 < 4$ ；但是选第一个和第二个可以，这时攻击力总和 $= 1 + 2 = 3$ ，防御力总和 $= 1$ ， $3 > 1$ 。

思考：到底按什么贪心？

朴素思考：看上去很像贪心，但是按照什么贪心呢？优先选防御最高的？还是攻击最高的？还是其它的什么？

思考：到底按什么贪心？

朴素思考：看上去很像贪心，但是按照什么贪心呢？优先选防御最高的？还是攻击最高的？还是其它的什么？

这个问题问得很好。这就是贪心解题的第一步也是最关键的一步：选择贪心策略。

思考：到底按什么贪心？

朴素思考：看上去很像贪心，但是按照什么贪心呢？优先选防御最高的？还是攻击最高的？还是其它的什么？

这个问题问得很好。这就是贪心解题的第一步也是最关键的一步：选择贪心策略。

当想出正确的贪心策略后，一般而言问题就迎刃而解了，只需要按照这个贪心策略依次选择最优的元素即可。

思考路径

关键区别

很多同学会把这一步和“按什么排序”混为一谈。

但注意：“每次买最便宜的”是策略，排序只是实现这个策略的手段。

先有“每次选价格最低的”这个决策，然后才考虑“怎么快速拿到价格最低的”——这恰好可以用排序。

如果改成一上来就问“按价格排序吗？”，就相当于跳过了“买最便宜的”这个策略思考，直接去问实现工具——这是本末倒置。

思考路径

正确的思考顺序：

思考路径

正确的思考顺序：

- 1 贪心策略：当前状态下，什么样的元素最优？

思考路径

正确的思考顺序：

- 1 贪心策略：当前状态下，什么样的元素最优？
- 2 实现工具：怎么快速拿到它？

正确做法

回到这个问题本身，什么样的贪心策略是正确的呢？

正确做法

回到这个问题本身，什么样的贪心策略是正确的呢？
先把所有道具的防御力加在一起，记作 S 。

正确做法

回到这个问题本身，什么样的贪心策略是正确的呢？

先把所有道具的防御力加在一起，记作 S 。

假设我们已经决定好了要选哪些道具作为攻击组。

正确做法

回到这个问题本身，什么样的贪心策略是正确的呢？

先把**所有道具的防御力加在一起**，记作 S 。

假设我们已经决定好了要选哪些道具作为攻击组。

那么剩下那些没被选中的道具，它们的防御力之和就是 S 减去被选中道具的防御力之和。

正确做法

回到这个问题本身，什么样的贪心策略是正确的呢？

先把**所有道具的防御力加在一起**，记作 S 。

假设我们已经决定好了要选哪些道具作为攻击组。

那么剩下那些没被选中的道具，它们的防御力之和就是 S 减去被选中道具的防御力之和。

题目要求被选中道具的攻击力之和大于没被选中道具的防御力之和，把上面那个式子代入这个条件中，可以得到：

$$\text{被选中道具的攻击力之和} > S - \text{被选中道具的防御力之和}$$

正确做法

回到这个问题本身，什么样的贪心策略是正确的呢？

先把**所有道具的防御力加在一起**，记作 S 。

假设我们已经决定好了要选哪些道具作为攻击组。

那么剩下那些没被选中的道具，它们的防御力之和就是 S 减去被选中道具的防御力之和。

题目要求被选中道具的攻击力之和大于没被选中道具的防御力之和，把上面那个式子代入这个条件中，可以得到：

被选中道具的攻击力之和 $> S -$ 被选中道具的防御力之和

把右边的防御力挪到左边：

被选中道具的攻击力之和 $+$ 被选中道具的防御力之和 $> S$

正确做法

回到这个问题本身，什么样的贪心策略是正确的呢？

先把所有道具的防御力加在一起，记作 S 。

假设我们已经决定好了要选哪些道具作为攻击组。

那么剩下那些没被选中的道具，它们的防御力之和就是 S 减去被选中道具的防御力之和。

题目要求被选中道具的攻击力之和大于没被选中道具的防御力之和，把上面那个式子代入这个条件中，可以得到：

$$\text{被选中道具的攻击力之和} > S - \text{被选中道具的防御力之和}$$

把右边的防御力挪到左边：

$$\text{被选中道具的攻击力之和} + \text{被选中道具的防御力之和} > S$$

所以每选中一个道具，它对“凑够 S ”的贡献就是它的攻击力 a_i 加上防御力 b_i 。

正确做法

问题变成了：

每个道具具有 $a_i + b_i$ 的权值，选尽量少的道具使权值之和大于 S 。

正确做法

问题变成了：

每个道具有 $a_i + b_i$ 的权值，选尽量少的道具使权值之和大于 S 。

要个数最少，就按权值从大到小排序，依次累加，直到超过 S 。

正确做法

问题变成了：

每个道具有 $a_i + b_i$ 的权值，选尽量少的道具使权值之和大于 S 。

要个数最少，就按权值从大到小排序，依次累加，直到超过 S 。
此时选取的个数就是答案。

反思

有没有豁然开朗的感觉？有想出来正确的策略吗？

反思

有没有豁然开朗的感觉？有想出来正确的策略吗？

事实上，这个贪心策略的思考方式对于初学者而言并不直观，我想有很多同学应该是没想到这个正确策略的，本人也没有想到怎么做。

反思

有没有豁然开朗的感觉？有想出来正确的策略吗？

事实上，这个贪心策略的思考方式对于初学者而言并不直观，我想有很多同学应该是没想到这个正确策略的，本人也没有想到怎么做。

然而不必气馁，想不到和想错是一件很正常的事情，特别是较为困难的贪心题目。大多数难的贪心题的考察点都在贪心策略的选择上，虽然有时候凭直觉猜结论往往能够猜得八九不离十，但是一道好的贪心题不应该让结论非常容易猜到，这时可能会想好几个错误结论后才能做对。

反思

既然不能猜到结论，遇到这类问题应该怎么办呢？

反思

既然不能猜到结论，遇到这类问题应该怎么办呢？

- 1 **首先，要猜但不能只猜。**直觉确实很重要，但是 ICPC 不是玄学的猜测大赛，每一个正确贪心策略都应该补上一个严谨的证明。我们将在第一部分看到一类贪心题目，这类题目有严谨的思考路径，但也常常借助先猜后证的想法。

反思

既然不能猜到结论，遇到这类问题应该怎么办呢？

- 1 首先，要猜但不能只猜。直觉确实很重要，但是 ICPC 不是玄学的猜测大赛，每一个正确贪心策略都应该补上一个严谨的证明。我们将在第一部分看到一类贪心题目，这类题目有严谨的思考路径，但也常常借助先猜后证的想法。
- 2 然后，可以尝试把它归类到一些常见模型上。这部分比较吃经验积累和转化问题的能力，需要慢慢积累。第二部分及之后我们会见到一些常见的模型。

反思

既然不能猜到结论，遇到这类问题应该怎么办呢？

- 1 首先，要猜但不能只猜。**直觉确实很重要，但是 ICPC 不是玄学的猜测大赛，每一个正确贪心策略都应该补上一个严谨的证明。我们将在第一部分看到一类贪心题目，这类题目有严谨的思考路径，但也常常借助先猜后证的想法。
- 2 然后，可以尝试把它归类到一些常见模型上。**这部分比较吃经验积累和转化问题的能力，需要慢慢积累。第二部分及之后我们会见到一些常见的模型。
- 3 最后，尝试找到朴素贪心策略错误的原因。**可以先写一个一定正确的做法（例如枚举每个元素取还是不取），然后尝试找出一组数据使得朴素的贪心策略出错，再看看正确做法的策略和贪心策略有什么不一样，找一找规律，分析贪心为什么失败了，从而找出真正正确的策略。

目录

- 1 引入
- 2 邻项交换
- 3 区间选点与区间覆盖
- 4 绝对值模型
- 5 哈夫曼模型
- 6 进阶技巧
- 7 总结

第一步：排队接水——任意交换

题目描述（P1223）

N 个人要去排队打水，第 i 个人打水需要 T_i 分钟。

现在你可以任意排列这些人的打水顺序，你需要使得所有人的总等待时间之和最小。

对于一个人，如果他是第一个去打水的，那么他不用等待，等待时间是 0；否则，他的等待时间是前面所有人打水时间的总和。

第一步：排队接水——任意交换

直觉：接水时间短的人应该排在前面。

第一步：排队接水——任意交换

直觉：接水时间短的人应该排在前面。
事实上，这确实是正确的贪心策略。

第一步：排队接水——任意交换

直觉：接水时间短的人应该排在前面。
事实上，这确实是正确的贪心策略。
但我们要证明它。怎么下手？

第一步：排队接水——任意交换

直觉：接水时间短的人应该排在前面。

事实上，这确实是正确的贪心策略。

但我们要证明它。怎么下手？

一个自然的想法是：任取两个不同的人 i 和 j ，如果他们的顺序是“反”的，也就是 i 在 j 前面但 i 的打水时间比 j 长，交换他们会不会让答案变好？

考虑两个人 i 和 j , 接水时间分别为 T_i 和 T_j , 有 $T_i > T_j$, 且当前顺序是 i 在前、 j 在后。假设它们中间隔着 K 个人。现在, 我们尝试交换 i 和 j , 看看这样能不能让总等待时间变小。

考虑两个人 i 和 j , 接水时间分别为 T_i 和 T_j , 有 $T_i > T_j$, 且当前顺序是 i 在前、 j 在后。假设它们中间隔着 K 个人。现在, 我们尝试交换 i 和 j , 看看这样能不能让总等待时间变小。

- 1 原来在 i 前面的人: 他前面既没有 i 也没有 j , 等待时间不变。

考虑两个人 i 和 j , 接水时间分别为 T_i 和 T_j , 有 $T_i > T_j$, 且当前顺序是 i 在前、 j 在后。假设它们中间隔着 K 个人。现在, 我们尝试交换 i 和 j , 看看这样能不能让总等待时间变小。

- 1 原来在 i 前面的人: 他前面既没有 i 也没有 j , 等待时间不变。
- 2 原来在 j 后面的人: 换不换前面都既有 i 又有 j , 等待时间不变。

考虑两个人 i 和 j , 接水时间分别为 T_i 和 T_j , 有 $T_i > T_j$, 且当前顺序是 i 在前、 j 在后。假设它们中间隔着 K 个人。现在, 我们尝试交换 i 和 j , 看看这样能不能让总等待时间变小。

- 1 原来在 i 前面的人: 他前面既没有 i 也没有 j , 等待时间不变。
- 2 原来在 j 后面的人: 换不换前面都既有 i 又有 j , 等待时间不变。
- 3 原来在 i 和 j 中间的人 (共 K 个): 他前面原来有 i , 现在换成了 j , 等待时间变化量是 $T_j - T_i$ 。

考虑两个人 i 和 j , 接水时间分别为 T_i 和 T_j , 有 $T_i > T_j$, 且当前顺序是 i 在前、 j 在后。假设它们中间隔着 K 个人。现在, 我们尝试交换 i 和 j , 看看这样能不能让总等待时间变小。

- 1 原来在 i 前面的人: 他前面既没有 i 也没有 j , 等待时间不变。
- 2 原来在 j 后面的人: 换不换前面都既有 i 又有 j , 等待时间不变。
- 3 原来在 i 和 j 中间的人 (共 K 个): 他前面原来有 i , 现在换成了 j , 等待时间变化量是 $T_j - T_i$ 。
- 4 i 和 j 自己的等待时间之和: i 前面人打水时间总和 + j 前面人打水时间总和, 第一项没有变; 对于第二项, 考虑 j 前面的唯一变化就是 i 换成了 j , 那么变化量也就是 $T_j - T_i$ 。因此这一项总变化量也是 $T_j - T_i$ 。

考虑两个人 i 和 j ，接水时间分别为 T_i 和 T_j ，有 $T_i > T_j$ ，且当前顺序是 i 在前、 j 在后。假设它们中间隔着 K 个人。现在，我们尝试交换 i 和 j ，看看这样能不能让总等待时间变小。

- 1 原来在 i 前面的人：他前面既没有 i 也没有 j ，等待时间不变。
- 2 原来在 j 后面的人：换不换前面都既有 i 又有 j ，等待时间不变。
- 3 原来在 i 和 j 中间的人（共 K 个）：他前面原来有 i ，现在换成了 j ，等待时间变化量是 $T_j - T_i$ 。
- 4 i 和 j 自己的等待时间之和： i 前面人打水时间总和 + j 前面人打水时间总和，第一项没有变；对于第二项，考虑 j 前面的唯一变化就是 i 换成了 j ，那么变化量也就是 $T_j - T_i$ 。因此这一项总变化量也是 $T_j - T_i$ 。
- 5 综上，如果交换 i 和 j ，会造成总等待时间的变化量是 $(K+1)(T_j - T_i)$ ，由于 $K \geq 0$ ， $T_i > T_j$ ，因此这个变化量是负数，那么交换会导致总等待时间变小。

贪心策略证明

现在再回过头来看我们的贪心策略：接水时间短的人应该排在前面，也就是按照接水时间从短到长来打水。

贪心策略证明

现在再回过头来看我们的贪心策略：接水时间短的人应该排在前面，也就是按照接水时间从短到长来打水。

考虑任意一个不是按接水时间从短到长排列的顺序。那么这个顺序里一定存在一对人 (i, j) ，满足 i 排在 j 前面，但 $T_i > T_j$ ——即“长的在前、短的在后”。根据我们刚才的结论，这对人交换后总等待时间会变小。

贪心策略证明

现在再回过头来看我们的贪心策略：接水时间短的人应该排在前面，也就是按照接水时间从短到长来打水。

考虑任意一个不是按接水时间从短到长排列的顺序。那么这个顺序里一定存在一对人 (i, j) ，满足 i 排在 j 前面，但 $T_i > T_j$ ——即“长的在前、短的在后”。根据我们刚才的结论，这对人交换后总等待时间会变小。

所以，只要顺序不是“短的在前、长的在后”，就一定还能通过交换让答案变得更优。

贪心策略证明

现在再回过头来看我们的贪心策略：接水时间短的人应该排在前面，也就是按照接水时间从短到长来打水。

考虑任意一个不是按接水时间从短到长排列的顺序。那么这个顺序里一定存在一对人 (i, j) ，满足 i 排在 j 前面，但 $T_i > T_j$ ——即“长的在前、短的在后”。根据我们刚才的结论，这对人交换后总等待时间会变小。

所以，只要顺序不是“短的在前、长的在后”，就一定还能通过交换让答案变得更优。

换句话说：最优解必须满足——不存在任何一对“长的在前、短的在后”的情况。

贪心策略证明

现在再回过头来看我们的贪心策略：接水时间短的人应该排在前面，也就是按照接水时间从短到长来打水。

考虑任意一个不是按接水时间从短到长排列的顺序。那么这个顺序里一定存在一对人 (i, j) ，满足 i 排在 j 前面，但 $T_i > T_j$ ——即“长的在前、短的在后”。根据我们刚才的结论，这对人交换后总等待时间会变小。

所以，只要顺序不是“短的在前、长的在后”，就一定还能通过交换让答案变得更优。

换句话说：最优解必须满足——不存在任何一对“长的在前、短的在后”的情况。

这恰好就是“按接水时间从短到长排列”的定义。

贪心策略证明

现在再回过头来看我们的贪心策略：接水时间短的人应该排在前面，也就是按照接水时间从短到长来打水。

考虑任意一个不是按接水时间从短到长排列的顺序。那么这个顺序里一定存在一对人 (i, j) ，满足 i 排在 j 前面，但 $T_i > T_j$ ——即“长的在前、短的在后”。根据我们刚才的结论，这对人交换后总等待时间会变小。

所以，只要顺序不是“短的在前、长的在后”，就一定还能通过交换让答案变得更优。

换句话说：最优解必须满足——不存在任何一对“长的在前、短的在后”的情况。

这恰好就是“按接水时间从短到长排列”的定义。

因此，贪心策略得证：最优顺序就是按 T_i 从小到大排序。

如何得出贪心策略

现在问题来了：这个贪心策略到底是怎么想出来的？
难道真的是凭空猜出“短的在前”吗？

如何得出贪心策略

现在问题来了：这个贪心策略到底是怎么想出来的？
难道真的是凭空猜出“短的在前”吗？
不是。这个策略不是猜出来的，是公式告诉我们的。

如何得出贪心策略

现在问题来了：这个贪心策略到底是怎么想出来的？

难道真的是凭空猜出“短的在前”吗？

不是。这个策略不是猜出来的，是公式告诉我们的。

再看一遍这个差值公式：

$$\text{原顺序代价} - \text{交换后代价} = (K + 1)(T_i - T_j)$$

如何得出贪心策略

现在问题来了：这个贪心策略到底是怎么想出来的？

难道真的是凭空猜出“短的在前”吗？

不是。这个策略不是猜出来的，是公式告诉我们的。

再看一遍这个差值公式：

$$\text{原顺序代价} - \text{交换后代价} = (K+1)(T_i - T_j)$$

这个公式本身就是一个判定器：

如何得出贪心策略

现在问题来了：这个贪心策略到底是怎么想出来的？

难道真的是凭空猜出“短的在前”吗？

不是。这个策略不是猜出来的，是公式告诉我们的。

再看一遍这个差值公式：

$$\text{原顺序代价} - \text{交换后代价} = (K+1)(T_i - T_j)$$

这个公式本身就是一个判定器：

- 把式子右边写成 $(K+1)(T_i - T_j)$;

如何得出贪心策略

现在问题来了：这个贪心策略到底是怎么想出来的？

难道真的是凭空猜出“短的在前”吗？

不是。这个策略不是猜出来的，是公式告诉我们的。

再看一遍这个差值公式：

$$\text{原顺序代价} - \text{交换后代价} = (K+1)(T_i - T_j)$$

这个公式本身就是一个判定器：

- 把式子右边写成 $(K+1)(T_i - T_j)$ ；
- 我们想让它 < 0 （这样不交换更优），那就需要 $T_i - T_j < 0$ ，即 $T_i < T_j$ ；

如何得出贪心策略

现在问题来了：这个贪心策略到底是怎么想出来的？
难道真的是凭空猜出“短的在前”吗？
不是。这个策略不是猜出来的，是公式告诉我们的。
再看一遍这个差值公式：

$$\text{原顺序代价} - \text{交换后代价} = (K+1)(T_i - T_j)$$

这个公式本身就是一个判定器：

- 把式子右边写成 $(K+1)(T_i - T_j)$;
- 我们想让它 < 0 （这样不交换更优），那就需要 $T_i - T_j < 0$ ，即 $T_i < T_j$;
- 也就是说，接水时间短的那个应该排在前面。

如何得出贪心策略

所以整个推理链条是这样的：

列式子 $\xrightarrow{\text{令差值} < 0}$ 得到比较条件 $T_i < T_j \xrightarrow{\text{翻译成策略}}$ 短的在前

如何得出贪心策略

所以整个推理链条是这样的：

列式子 $\xrightarrow{\text{令差值}<0}$ 得到比较条件 $T_i < T_j \xrightarrow{\text{翻译成策略}}$ 短的在前

策略是从公式里推导出来的，不是猜出来的。

这就是邻项交换的核心思想：交换两个人，列出代价变化，令其小于 0，反推出谁应该在谁前面。

第二步：任意交换开始变复杂了

刚才我们很幸运：排队接水里，任意两个人交换后，中间一堆贡献全部抵消了，差值只跟 T_i 和 T_j 有关，结论干净利落。

第二步：任意交换开始变复杂了

刚才我们很幸运：排队接水里，任意两个人交换后，中间一堆贡献全部抵消了，差值只跟 T_i 和 T_j 有关，结论干净利落。

于是我们可能会想：是不是所有排序贪心题都可以用同样的方式——任取两个人交换，列式子，直接推出比较规则？

第二步：任意交换开始变复杂了

刚才我们很幸运：排队接水里，任意两个人交换后，中间一堆贡献全部抵消了，差值只跟 T_i 和 T_j 有关，结论干净利落。

于是我们可能会想：是不是所有排序贪心题都可以用同样的方式——任取两个人交换，列式子，直接推出比较规则？

为了回答这个问题，我们来看另一道经典题。

第二步：任意交换开始变复杂了

题目描述（P8113 [Cnoi2021] 自我主义的平衡者）

有 n 个人评分，平台最高分为 m 。第 i 个人的心理预期分数为 E_i 。

你需要构造一种打分顺序，使得最终平均分最大/最小。
轮到第 i 个人打分时，他的评分规则如下：

第二步：任意交换开始变复杂了

题目描述（P8113 [Cnoi2021] 自我主义的平衡者）

有 n 个人评分，平台最高分为 m 。第 i 个人的心理预期分数为 E_i 。

你需要构造一种打分顺序，使得最终平均分最大/最小。
轮到第 i 个人打分时，他的评分规则如下：

- 如果当前平均分严格高于 E_i ，则打 0 分；

第二步：任意交换开始变复杂了

题目描述（P8113 [Cnoi2021] 自我主义的平衡者）

有 n 个人评分，平台最高分为 m 。第 i 个人的心理预期分数为 E_i 。

你需要构造一种打分顺序，使得最终平均分最大/最小。
轮到第 i 个人打分时，他的评分规则如下：

- 如果当前平均分严格高于 E_i ，则打 0 分；
- 否则（平均分 $\leq E_i$ ），打 m 分。

尝试任意交换

这题看起来不大一样——每个人的分数不是固定的，而是取决于当前平均分。

尝试任意交换

这题看起来不大一样——每个人的分数不是固定的，而是取决于当前平均分。
我们还是用刚刚学的交换方法来推一下。

尝试任意交换

假设当前已经有 k 个人评完分，当前平均分为 $\bar{x}$ 。

尝试任意交换

假设当前已经有 k 个人评完分，当前平均分为 $\bar{x}$ 。

考虑接下来两个人 i 和 j ，预期分数分别为 E_i 和 E_j 。

尝试任意交换

假设当前已经有 k 个人评完分，当前平均分为 $\bar{x}$ 。

考虑接下来两个人 i 和 j ，预期分数分别为 E_i 和 E_j 。

第一个人（假设是 i ）的得分取决于 $\bar{x}$ 与 E_i 的大小关系：

$$\text{score}(i) = \begin{cases} 0, & \bar{x} > E_i \\ m, & \bar{x} \leq E_i \end{cases}$$

尝试任意交换

假设当前已经有 k 个人评完分，当前平均分为 $\bar{x}$ 。

考虑接下来两个人 i 和 j ，预期分数分别为 E_i 和 E_j 。

第一个人（假设是 i ）的得分取决于 $\bar{x}$ 与 E_i 的大小关系：

$$\text{score}(i) = \begin{cases} 0, & \bar{x} > E_i \\ m, & \bar{x} \leq E_i \end{cases}$$

第二个人（ j ）的得分取决于更新后的平均分：

$$\bar{x}' = \frac{k\bar{x} + \text{score}(i)}{k+1}$$

尝试任意交换

假设当前已经有 k 个人评完分，当前平均分为 $\bar{x}$ 。

考虑接下来两个人 i 和 j ，预期分数分别为 E_i 和 E_j 。

第一个人（假设是 i ）的得分取决于 $\bar{x}$ 与 E_i 的大小关系：

$$\text{score}(i) = \begin{cases} 0, & \bar{x} > E_i \\ m, & \bar{x} \leq E_i \end{cases}$$

第二个人（ j ）的得分取决于更新后的平均分：

$$\bar{x}' = \frac{k\bar{x} + \text{score}(i)}{k+1}$$

这里有一个关键问题：如果 i 和 j 中间还隔着其他人，每个人的得分都会受到前面所有人分数的影响。

尝试任意交换

假设当前已经有 k 个人评完分，当前平均分为 $\bar{x}$ 。

考虑接下来两个人 i 和 j ，预期分数分别为 E_i 和 E_j 。

第一个人（假设是 i ）的得分取决于 $\bar{x}$ 与 E_i 的大小关系：

$$\text{score}(i) = \begin{cases} 0, & \bar{x} > E_i \\ m, & \bar{x} \leq E_i \end{cases}$$

第二个人（ j ）的得分取决于更新后的平均分：

$$\bar{x}' = \frac{k\bar{x} + \text{score}(i)}{k+1}$$

这里有一个关键问题：如果 i 和 j 中间还隔着其他人，每个人的得分都会受到前面所有人分数的影响。

这意味着什么？

任意交换为什么变复杂了

假设 i 和 j 中间隔着 K 个人。

任意交换为什么变复杂了

假设 i 和 j 中间隔着 K 个人。

把 i 和 j 对调，中间那 K 个人面对的“当前平均分”会完全改变。

任意交换为什么变复杂了

假设 i 和 j 中间隔着 K 个人。

把 i 和 j 对调，中间那 K 个人面对的“当前平均分”会完全改变。
因为 i 和 j 的分数可能不同（一个打 0，一个打 m ），导致中间每个人面前的平均分都变了。

任意交换为什么变复杂了

假设 i 和 j 中间隔着 K 个人。

把 i 和 j 对调，中间那 K 个人面对的“当前平均分”会完全改变。因为 i 和 j 的分数可能不同（一个打 0，一个打 m ），导致中间每个人面前的平均分都变了。

于是，中间每个人的得分可能从 0 变成 m ，或从 m 变成 0；这些变化消不掉，无法像排队接水那样直接抵消；最终差值公式会变得极其复杂。

任意交换为什么变复杂了

假设 i 和 j 中间隔着 K 个人。

把 i 和 j 对调，中间那 K 个人面对的“当前平均分”会完全改变。因为 i 和 j 的分数可能不同（一个打 0，一个打 m ），导致中间每个人面前的平均分都变了。

于是，中间每个人的得分可能从 0 变成 m ，或从 m 变成 0；

这些变化消不掉，无法像排队接水那样直接抵消；

最终差值公式会变得极其复杂。

这就是“任意交换”失效的地方。

任意交换为什么变复杂了

假设 i 和 j 中间隔着 K 个人。

把 i 和 j 对调，中间那 K 个人面对的“当前平均分”会完全改变。因为 i 和 j 的分数可能不同（一个打 0，一个打 m ），导致中间每个人面前的平均分都变了。

于是，中间每个人的得分可能从 0 变成 m ，或从 m 变成 0；

这些变化消不掉，无法像排队接水那样直接抵消；

最终差值公式会变得极其复杂。

这就是“任意交换”失效的地方。

那怎么办？——退到相邻交换。

退到相邻交换

由于最终平均分最大和最小两个问题的过程十分相似，这里只考虑**最终满分人数最多**的情况（即平均分最大）。

退到相邻交换

由于最终平均分最大和最小两个问题的过程十分相似，这里只考虑**最终满分人数最多**的情况（即平均分最大）。

不妨设有两个相邻的人 x 和 y ，他们的预期值分别为 E_x 和 E_y ，且满足 $E_x \leq E_y$ 。

退到相邻交换

由于最终平均分最大和最小两个问题的过程十分相似，这里只考虑**最终满分人数最多**的情况（即平均分最大）。

不妨设有两个相邻的人 x 和 y ，他们的预期值分别为 E_x 和 E_y ，且满足 $E_x \leq E_y$ 。

设这两个人打分前的平均分为 S ，第一个人打分后的平均分为 S' 。

退到相邻交换

由于最终平均分最大和最小两个问题的过程十分相似，这里只考虑**最终满分人数最多**的情况（即平均分最大）。

不妨设有两个相邻的人 x 和 y ，他们的预期值分别为 E_x 和 E_y ，且满足 $E_x \leq E_y$ 。

设这两个人打分前的平均分为 S ，第一个人打分后的平均分为 S' 。

接下来根据 S 与 E_x 的大小关系分类讨论。

情况一: $S \leq E_x$

若 $S \leq E_x$, 则 x 会打一个满分。

情况一: $S \leq E_x$

若 $S \leq E_x$, 则 x 会打一个满分。

打分后的平均分为 S' 。此时 S' 与 E_y 的大小关系有两种可能:

情况一: $S \leq E_x$

若 $S \leq E_x$, 则 x 会打一个满分。

打分后的平均分为 S' 。此时 S' 与 E_y 的大小关系有两种可能:

情况 1.1: $S' \leq E_y$

情况一: $S \leq E_x$

若 $S \leq E_x$, 则 x 会打一个满分。

打分后的平均分为 S' 。此时 S' 与 E_y 的大小关系有两种可能:

情况 1.1: $S' \leq E_y$

- y 会打一个满分, 两人贡献为 **2** 个满分。

情况一: $S \leq E_x$

若 $S \leq E_x$, 则 x 会打一个满分。

打分后的平均分为 S' 。此时 S' 与 E_y 的大小关系有两种可能:

情况 1.1: $S' \leq E_y$

- y 会打一个满分, 两人贡献为 **2** 个满分。
- 若交换 x 和 y 的顺序:

情况一: $S \leq E_x$

若 $S \leq E_x$, 则 x 会打一个满分。

打分后的平均分为 S' 。此时 S' 与 E_y 的大小关系有两种可能:

情况 1.1: $S' \leq E_y$

- y 会打一个满分, 两人贡献为 **2** 个满分。
- 若交换 x 和 y 的顺序:
 - 由于 $E_x \leq E_y$, 有 $S \leq E_y$, 所以 y 先打满分;

情况一: $S \leq E_x$

若 $S \leq E_x$, 则 x 会打一个满分。

打分后的平均分为 S' 。此时 S' 与 E_y 的大小关系有两种可能:

情况 1.1: $S' \leq E_y$

- y 会打一个满分, 两人贡献为 **2** 个满分。
- 若交换 x 和 y 的顺序:
 - 由于 $E_x \leq E_y$, 有 $S \leq E_y$, 所以 y 先打满分;
 - 交换后 x 面临的平均分与原来的 S' 相同, 但 S' 与 E_x 的大小关系不确定;

情况一: $S \leq E_x$

若 $S \leq E_x$, 则 x 会打一个满分。

打分后的平均分为 S' 。此时 S' 与 E_y 的大小关系有两种可能:

情况 1.1: $S' \leq E_y$

- y 会打一个满分, 两人贡献为 **2** 个满分。
- 若交换 x 和 y 的顺序:
 - 由于 $E_x \leq E_y$, 有 $S \leq E_y$, 所以 y 先打满分;
 - 交换后 x 面临的平均分与原来的 S' 相同, 但 S' 与 E_x 的大小关系不确定;
 - 所以 x 可能打 0 分, 也可能打满分。不会更优。

情况一: $S \leq E_x$

情况 1.2: $S' > E_y$

情况一: $S \leq E_x$

情况 1.2: $S' > E_y$

- y 会打 0 分, 两人贡献为 1 个满分 + 1 个 0 分。

情况一: $S \leq E_x$

情况 1.2: $S' > E_y$

- y 会打 0 分, 两人贡献为 1 个满分 + 1 个 0 分。
- 若交换 x 和 y 的顺序:

情况一: $S \leq E_x$

情况 1.2: $S' > E_y$

- y 会打 0 分, 两人贡献为 1 个满分 + 1 个 0 分。
- 若交换 x 和 y 的顺序:
 - 由于 $S \leq E_y$, y 先打满分;

情况一: $S \leq E_x$

情况 1.2: $S' > E_y$

- y 会打 0 分, 两人贡献为 1 个满分 + 1 个 0 分。
- 若交换 x 和 y 的顺序:
 - 由于 $S \leq E_y$, y 先打满分;
 - x 面临的平均分仍为 S' , 且 $S' > E_y \geq E_x$, 所以 x 打 0 分。

情况一: $S \leq E_x$

情况 1.2: $S' > E_y$

- y 会打 0 分, 两人贡献为 1 个满分 + 1 个 0 分。
- 若交换 x 和 y 的顺序:
 - 由于 $S \leq E_y$, y 先打满分;
 - x 面临的平均分仍为 S' , 且 $S' > E_y \geq E_x$, 所以 x 打 0 分。
- 交换后贡献仍为 1 个满分 + 1 个 0 分。对答案没有影响。

情况二: $S > E_x$

若 $S > E_x$, 则 x 会打 0 分。

情况二： $S > E_x$

若 $S > E_x$ ，则 x 会打 0 分。

打分后的平均分为 S' 。同样分两种情况讨论：

情况二： $S > E_x$

若 $S > E_x$ ，则 x 会打 0 分。

打分后的平均分为 S' 。同样分两种情况讨论：

情况 2.1： $S' \leq E_y$

情况二: $S > E_x$

若 $S > E_x$, 则 x 会打 0 分。

打分后的平均分为 S' 。同样分两种情况讨论:

情况 2.1: $S' \leq E_y$

- y 会打满分, 两人贡献为 1 个 0 分 + 1 个满分。

情况二： $S > E_x$

若 $S > E_x$ ，则 x 会打 0 分。

打分后的平均分为 S' 。同样分两种情况讨论：

情况 2.1： $S' \leq E_y$

- y 会打满分，两人贡献为 1 个 0 分 + 1 个满分。
- 若交换 x 和 y 的顺序，分两种子情况：

情况二： $S > E_x$

若 $S > E_x$ ，则 x 会打 0 分。

打分后的平均分为 S' 。同样分两种情况讨论：

情况 2.1： $S' \leq E_y$

- y 会打满分，两人贡献为 1 个 0 分 + 1 个满分。
- 若交换 x 和 y 的顺序，分两种子情况：
 - 若 $S \leq E_y$ ：则 y 先打满分。此时 $S' \geq S > E_x$ ，所以 x 打 0 分。贡献不变。

情况二: $S > E_x$

若 $S > E_x$, 则 x 会打 0 分。

打分后的平均分为 S' 。同样分两种情况讨论:

情况 2.1: $S' \leq E_y$

- y 会打满分, 两人贡献为 1 个 0 分 + 1 个满分。
- 若交换 x 和 y 的顺序, 分两种子情况:
 - 若 $S \leq E_y$: 则 y 先打满分。此时 $S' \geq S > E_x$, 所以 x 打 0 分。贡献不变。
 - 若 $S > E_y$: 则 y 先打 0 分。即使交换后 $S' \leq E_x$, 也只能得到 1 个满分 + 1 个 0 分, 不可能更优。

情况二: $S > E_x$

若 $S > E_x$, 则 x 会打 0 分。

打分后的平均分为 S' 。同样分两种情况讨论:

情况 2.1: $S' \leq E_y$

- y 会打满分, 两人贡献为 1 个 0 分 + 1 个满分。
- 若交换 x 和 y 的顺序, 分两种子情况:
 - 若 $S \leq E_y$: 则 y 先打满分。此时 $S' \geq S > E_x$, 所以 x 打 0 分。贡献不变。
 - 若 $S > E_y$: 则 y 先打 0 分。即使交换后 $S' \leq E_x$, 也只能得到 1 个满分 + 1 个 0 分, 不可能更优。
- **结论: 交换不会使答案更优。**

情况二: $S > E_x$

情况 2.2: $S' > E_y$

情况二: $S > E_x$

情况 2.2: $S' > E_y$

- y 会打 0 分, 两人贡献为 2 个 0 分。

情况二: $S > E_x$

情况 2.2: $S' > E_y$

- y 会打 0 分, 两人贡献为 **2 个 0 分**。
- 若交换 x 和 y 的顺序:

情况二: $S > E_x$

情况 2.2: $S' > E_y$

- y 会打 0 分, 两人贡献为 **2 个 0 分**。
- 若交换 x 和 y 的顺序:
 - 由于 $S > S' > E_y \geq E_x$, 两人都会打 0 分。

情况二: $S > E_x$

情况 2.2: $S' > E_y$

- y 会打 0 分, 两人贡献为 **2 个 0 分**。
- 若交换 x 和 y 的顺序:
 - 由于 $S > S' > E_y \geq E_x$, 两人都会打 0 分。
- 交换后贡献仍为 2 个 0 分。

情况二: $S > E_x$

情况 2.2: $S' > E_y$

- y 会打 0 分, 两人贡献为 **2 个 0 分**。
- 若交换 x 和 y 的顺序:
 - 由于 $S > S' > E_y \geq E_x$, 两人都会打 0 分。
- 交换后贡献仍为 2 个 0 分。
- **结论: 交换对答案没有影响。**

证明的结论

我们分类讨论了所有可能的情况：

条件	交换后贡献变化	结论
$S \leq E_x, S' \leq E_y$	变差或不变	交换不优
$S \leq E_x, S' > E_y$	不变	交换无影响
$S > E_x, S' \leq E_y$	变差或不变	交换不优
$S > E_x, S' > E_y$	不变	交换无影响

证明的结论

我们分类讨论了所有可能的情况：

条件	交换后贡献变化	结论
$S \leq E_x, S' \leq E_y$	变差或不变	交换不优
$S \leq E_x, S' > E_y$	不变	交换无影响
$S > E_x, S' \leq E_y$	变差或不变	交换不优
$S > E_x, S' > E_y$	不变	交换无影响

综合结论：

证明的结论

我们分类讨论了所有可能的情况：

条件	交换后贡献变化	结论
$S \leq E_x, S' \leq E_y$	变差或不变	交换不优
$S \leq E_x, S' > E_y$	不变	交换无影响
$S > E_x, S' \leq E_y$	变差或不变	交换不优
$S > E_x, S' > E_y$	不变	交换无影响

综合结论：

对于任意两个人 x, y ，只要 $E_x \leq E_y$ ，那么 x 排在 y 前面，一定
不会比 y 排在 x 前面更差。

证明的结论

我们分类讨论了所有可能的情况：

条件	交换后贡献变化	结论
$S \leq E_x, S' \leq E_y$	变差或不变	交换不优
$S \leq E_x, S' > E_y$	不变	交换无影响
$S > E_x, S' \leq E_y$	变差或不变	交换不优
$S > E_x, S' > E_y$	不变	交换无影响

综合结论：

对于任意两个人 x, y ，只要 $E_x \leq E_y$ ，那么 x 排在 y 前面，一定
不会比 y 排在 x 前面更差。

也就是说，预期分数小的人排在前面，是最优策略。

第二步小结：为什么要退到相邻交换

通过这道题，我们看到了邻项交换的完整动机：

第二步小结：为什么要退到相邻交换

通过这道题，我们看到了邻项交换的完整动机：

- 排队接水（第一步）：任意交换式子干净，直接推出规则；

第二步小结：为什么要退到相邻交换

通过这道题，我们看到了邻项交换的完整动机：

- 排队接水（第一步）：任意交换式子干净，直接推出规则；
- 平衡者（第二步）：任意交换式子太复杂（中间人的状态会变），被迫退到相邻交换；

第二步小结：为什么要退到相邻交换

通过这道题，我们看到了邻项交换的完整动机：

- 排队接水（第一步）：任意交换式子干净，直接推出规则；
- 平衡者（第二步）：任意交换式子太复杂（中间人的状态会变），被迫退到相邻交换；
- 相邻交换中间没人，式子可算，推出比较规则；

第二步小结：为什么要退到相邻交换

所以“退到相邻交换”不是退步，而是在复杂问题中唯一可行的前进方向。

第二步小结：为什么要退到相邻交换

所以“退到相邻交换”不是退步，而是在复杂问题中唯一可行的前进方向。

但既然如此，为什么不所有题目都只考虑邻项交换呢？以及，邻项交换为什么正确呢？——这就是第三步要讲的内容。

第三步：邻项交换注意点

CF2024C / CF2023A

给定 N 个长度为 2 的数组 $[a_i, b_i]$ 。你需要将它们按某种顺序拼接成一个长度为 $2N$ 的数组，使得最终数组中的**逆序对数**最小。

第三步：邻项交换注意点

CF2024C / CF2023A

给定 N 个长度为 2 的数组 $[a_i, b_i]$ 。你需要将它们按某种顺序拼接成一个长度为 $2N$ 的数组，使得最终数组中的**逆序对数**最小。

这是一道标准的邻项交换题。按套路来：

第三步：邻项交换注意点

CF2024C / CF2023A

给定 N 个长度为 2 的数组 $[a_i, b_i]$ 。你需要将它们按某种顺序拼接成一个长度为 $2N$ 的数组，使得最终数组中的**逆序对数**最小。

这是一道标准的邻项交换题。按套路来：
设相邻两项为 $X = [a, b]$ 和 $Y = [c, d]$ 。

第三步：邻项交换注意点

CF2024C / CF2023A

给定 N 个长度为 2 的数组 $[a_i, b_i]$ 。你需要将它们按某种顺序拼接成一个长度为 $2N$ 的数组，使得最终数组中的逆序对数最小。

这是一道标准的邻项交换题。按套路来：

设相邻两项为 $X = [a, b]$ 和 $Y = [c, d]$ 。

考虑 X 在前、 Y 在后，与 Y 在前、 X 在后，跨项逆序对的数量差异。

第三步：邻项交换注意点

CF2024C / CF2023A

给定 N 个长度为 2 的数组 $[a_i, b_i]$ 。你需要将它们按某种顺序拼接成一个长度为 $2N$ 的数组，使得最终数组中的**逆序对数**最小。

这是一道标准的邻项交换题。按套路来：

设相邻两项为 $X = [a, b]$ 和 $Y = [c, d]$ 。

考虑 X 在前、 Y 在后，与 Y 在前、 X 在后，跨项逆序对的数量差异。

X 内部和 Y 内部的逆序对不受顺序影响，只需要比较跨项部分。

列相邻式子

$$X = [a, b], \quad Y = [c, d]。$$

列相邻式子

$X = [a, b]$, $Y = [c, d]$ 。

X 在前, Y 在后: 跨项逆序对 = 从 X 中选一个数、从 Y 中选一个数, 前者大于后者。

$$(a > c) + (a > d) + (b > c) + (b > d)$$

列相邻式子

$$X = [a, b], Y = [c, d].$$

X 在前, Y 在后: 跨项逆序对 = 从 X 中选一个数、从 Y 中选一个数, 前者大于后者。

$$(a > c) + (a > d) + (b > c) + (b > d)$$

Y 在前, X 在后:

$$(c > a) + (c > b) + (d > a) + (d > b)$$

列相邻式子

$$X = [a, b], Y = [c, d].$$

X 在前, Y 在后: 跨项逆序对 = 从 X 中选一个数、从 Y 中选一个数, 前者大于后者。

$$(a > c) + (a > d) + (b > c) + (b > d)$$

Y 在前, X 在后:

$$(c > a) + (c > b) + (d > a) + (d > b)$$

所以 X 应排在 Y 前面, 当且仅当:

$$(a > c) + (a > d) + (b > c) + (b > d) < (c > a) + (c > b) + (d > a) + (d > b)$$

列相邻式子

$$X = [a, b], Y = [c, d].$$

X 在前, Y 在后: 跨项逆序对 = 从 X 中选一个数、从 Y 中选一个数, 前者大于后者。

$$(a > c) + (a > d) + (b > c) + (b > d)$$

Y 在前, X 在后:

$$(c > a) + (c > b) + (d > a) + (d > b)$$

所以 X 应排在 Y 前面, 当且仅当:

$$(a > c) + (a > d) + (b > c) + (b > d) < (c > a) + (c > b) + (d > a) + (d > b)$$

把这个作为比较规则交上去——**WA**。

列相邻式子

$$X = [a, b], Y = [c, d].$$

X 在前, Y 在后: 跨项逆序对 = 从 X 中选一个数、从 Y 中选一个数, 前者大于后者。

$$(a > c) + (a > d) + (b > c) + (b > d)$$

Y 在前, X 在后:

$$(c > a) + (c > b) + (d > a) + (d > b)$$

所以 X 应排在 Y 前面, 当且仅当:

$$(a > c) + (a > d) + (b > c) + (b > d) < (c > a) + (c > b) + (d > a) + (d > b)$$

把这个作为比较规则交上去——**WA**。

为什么? 我们来构造一组数据。

反例构造

取三组数据：

$$A = (0, 4), \quad B = (1, 3), \quad C = (0, 5)$$

反例构造

取三组数据：

$$A = (0, 4), \quad B = (1, 3), \quad C = (0, 5)$$

先定义“等号”：我们说 $X = Y$ ，当且仅当 X 既不小于 Y ， Y 也不小于 X ，即：

$$X = Y \iff \text{not } (X < Y) \text{ and not } (Y < X)$$

反例构造

取三组数据：

$$A = (0, 4), \quad B = (1, 3), \quad C = (0, 5)$$

先定义“等号”：我们说 $X = Y$ ，当且仅当 X 既不小于 Y ， Y 也不小于 X ，即：

$$X = Y \iff \text{not } (X < Y) \text{ and not } (Y < X)$$

其中 $X < Y$ 表示 X 应排在 Y 前面，由我们的比较规则定义。如果 $X = Y$ 那么 X 和 Y 谁排在前面理应无所谓，可以类比买东西：两个价格一样的物品，先买哪个没有影响。

反例构造

取三组数据：

$$A = (0, 4), \quad B = (1, 3), \quad C = (0, 5)$$

先定义“等号”：我们说 $X = Y$ ，当且仅当 X 既不小于 Y ， Y 也不小于 X ，即：

$$X = Y \iff \text{not } (X < Y) \text{ and not } (Y < X)$$

其中 $X < Y$ 表示 X 应排在 Y 前面，由我们的比较规则定义。如果 $X = Y$ 那么 X 和 Y 谁排在前面理应无所谓，可以类比买东西：两个价格一样的物品，先买哪个没有影响。
接下来手动计算这三组数据之间的比较关系。

计算 A 与 B 的关系

$$A = (0, 4), B = (1, 3)。$$

计算 A 与 B 的关系

$A = (0, 4)$, $B = (1, 3)$ 。

A 在 B 前面时, 跨项逆序对数:

$$(0 > 1) + (0 > 3) + (4 > 1) + (4 > 3) = 0 + 0 + 1 + 1 = 2$$

计算 A 与 B 的关系

$A = (0, 4)$, $B = (1, 3)$ 。

A 在 B 前面时，跨项逆序对数：

$$(0 > 1) + (0 > 3) + (4 > 1) + (4 > 3) = 0 + 0 + 1 + 1 = 2$$

B 在 A 前面时，跨项逆序对数：

$$(1 > 0) + (1 > 4) + (3 > 0) + (3 > 4) = 1 + 0 + 1 + 0 = 2$$

计算 A 与 B 的关系

$A = (0, 4)$, $B = (1, 3)$ 。

A 在 B 前面时, 跨项逆序对数:

$$(0 > 1) + (0 > 3) + (4 > 1) + (4 > 3) = 0 + 0 + 1 + 1 = 2$$

B 在 A 前面时, 跨项逆序对数:

$$(1 > 0) + (1 > 4) + (3 > 0) + (3 > 4) = 1 + 0 + 1 + 0 = 2$$

两边相等, 所以:

$$\text{cmp}(A, B) = \text{假}, \quad \text{cmp}(B, A) = \text{假}$$

计算 A 与 B 的关系

$A = (0, 4)$, $B = (1, 3)$ 。

A 在 B 前面时, 跨项逆序对数:

$$(0 > 1) + (0 > 3) + (4 > 1) + (4 > 3) = 0 + 0 + 1 + 1 = 2$$

B 在 A 前面时, 跨项逆序对数:

$$(1 > 0) + (1 > 4) + (3 > 0) + (3 > 4) = 1 + 0 + 1 + 0 = 2$$

两边相等, 所以:

$$\text{cmp}(A, B) = \text{假}, \quad \text{cmp}(B, A) = \text{假}$$

根据等号的定义:

$A = B$

计算 B 与 C 的关系

$$B = (1, 3), \quad C = (0, 5)。$$

计算 B 与 C 的关系

$B = (1, 3)$, $C = (0, 5)$ 。

B 在 C 前面时, 跨项逆序对数:

$$(1 > 0) + (1 > 5) + (3 > 0) + (3 > 5) = 1 + 0 + 1 + 0 = 2$$

计算 B 与 C 的关系

$B = (1, 3)$, $C = (0, 5)$ 。

B 在 C 前面时, 跨项逆序对数:

$$(1 > 0) + (1 > 5) + (3 > 0) + (3 > 5) = 1 + 0 + 1 + 0 = 2$$

C 在 B 前面时, 跨项逆序对数:

$$(0 > 1) + (0 > 3) + (5 > 1) + (5 > 3) = 0 + 0 + 1 + 1 = 2$$

计算 B 与 C 的关系

$B = (1, 3)$, $C = (0, 5)$ 。

B 在 C 前面时, 跨项逆序对数:

$$(1 > 0) + (1 > 5) + (3 > 0) + (3 > 5) = 1 + 0 + 1 + 0 = 2$$

C 在 B 前面时, 跨项逆序对数:

$$(0 > 1) + (0 > 3) + (5 > 1) + (5 > 3) = 0 + 0 + 1 + 1 = 2$$

两边相等, 所以:

$$\text{cmp}(B, C) = \text{假}, \quad \text{cmp}(C, B) = \text{假}$$

计算 B 与 C 的关系

$B = (1, 3)$, $C = (0, 5)$ 。

B 在 C 前面时, 跨项逆序对数:

$$(1 > 0) + (1 > 5) + (3 > 0) + (3 > 5) = 1 + 0 + 1 + 0 = 2$$

C 在 B 前面时, 跨项逆序对数:

$$(0 > 1) + (0 > 3) + (5 > 1) + (5 > 3) = 0 + 0 + 1 + 1 = 2$$

两边相等, 所以:

$$\text{cmp}(B, C) = \text{假}, \quad \text{cmp}(C, B) = \text{假}$$

根据等号的定义:

$$\boxed{B = C}$$

计算 A 与 C 的关系

$$A = (0, 4), \quad C = (0, 5)。$$

计算 A 与 C 的关系

$A = (0, 4)$, $C = (0, 5)$ 。

A 在 C 前面时, 跨项逆序对数:

$$(0 > 0) + (0 > 5) + (4 > 0) + (4 > 5) = 0 + 0 + 1 + 0 = 1$$

计算 A 与 C 的关系

$A = (0, 4)$, $C = (0, 5)$ 。

A 在 C 前面时，跨项逆序对数：

$$(0 > 0) + (0 > 5) + (4 > 0) + (4 > 5) = 0 + 0 + 1 + 0 = 1$$

C 在 A 前面时，跨项逆序对数：

$$(0 > 0) + (0 > 4) + (5 > 0) + (5 > 4) = 0 + 0 + 1 + 1 = 2$$

计算 A 与 C 的关系

$A = (0, 4)$, $C = (0, 5)$ 。

A 在 C 前面时, 跨项逆序对数:

$$(0 > 0) + (0 > 5) + (4 > 0) + (4 > 5) = 0 + 0 + 1 + 0 = 1$$

C 在 A 前面时, 跨项逆序对数:

$$(0 > 0) + (0 > 4) + (5 > 0) + (5 > 4) = 0 + 0 + 1 + 1 = 2$$

左边 $1 < 2$, 所以:

$$\text{cmp}(A, C) = \text{真}$$

计算 A 与 C 的关系

$A = (0, 4)$, $C = (0, 5)$ 。

A 在 C 前面时, 跨项逆序对数:

$$(0 > 0) + (0 > 5) + (4 > 0) + (4 > 5) = 0 + 0 + 1 + 0 = 1$$

C 在 A 前面时, 跨项逆序对数:

$$(0 > 0) + (0 > 4) + (5 > 0) + (5 > 4) = 0 + 0 + 1 + 1 = 2$$

左边 $1 < 2$, 所以:

$$\text{cmp}(A, C) = \text{真}$$

也就是说:

$$A < C$$

传递性被破坏了

把三页的计算结果放在一起：

$$A = B, \quad B = C, \quad \text{但} \quad A < C$$

传递性被破坏了

把三页的计算结果放在一起：

$$A = B, \quad B = C, \quad \text{但} \quad A < C$$

这违反了什么？等号的传递性。

传递性被破坏了

把三页的计算结果放在一起：

$$A = B, \quad B = C, \quad \text{但} \quad A < C$$

这违反了什么？等号的传递性。

如果 $A = B$ 且 $B = C$ ，那么必须要有 $A = C$ 。

传递性被破坏了

把三页的计算结果放在一起：

$$A = B, \quad B = C, \quad \text{但} \quad A < C$$

这违反了什么？等号的传递性。

如果 $A = B$ 且 $B = C$ ，那么必须要有 $A = C$ 。

但我们得到的是 $A < C$ ，这是矛盾的。

传递性被破坏了

把三页的计算结果放在一起：

$$A = B, \quad B = C, \quad \text{但} \quad A < C$$

这违反了什么？等号的传递性。

如果 $A = B$ 且 $B = C$ ，那么必须要有 $A = C$ 。

但我们得到的是 $A < C$ ，这是矛盾的。

这就好比说 A 和 B 一样重， B 和 C 一样重，结果 A 比 C 轻——荒谬。

传递性被破坏了

把三页的计算结果放在一起：

$$A = B, \quad B = C, \quad \text{但} \quad A < C$$

这违反了什么？等号的传递性。

如果 $A = B$ 且 $B = C$ ，那么必须要有 $A = C$ 。

但我们得到的是 $A < C$ ，这是矛盾的。

这就好比说 A 和 B 一样重， B 和 C 一样重，结果 A 比 C 轻——荒谬。

结论：这个比较规则不满足严格弱序中“等价关系的传递性”这一要求。

传递性被破坏了

把三页的计算结果放在一起：

$$A = B, \quad B = C, \quad \text{但} \quad A < C$$

这违反了什么？等号的传递性。

如果 $A = B$ 且 $B = C$ ，那么必须要有 $A = C$ 。

但我们得到的是 $A < C$ ，这是矛盾的。

这就好比说 A 和 B 一样重， B 和 C 一样重，结果 A 比 C 轻——荒谬。

结论：这个比较规则不满足严格弱序中“等价关系的传递性”这一要求。

而 `std::sort` 要求比较器必须满足严格弱序，否则行为是未定义的——可能乱排、可能死循环，WA 就是这么来的。

修复方案

正确策略：按 $(\min(a, b), \max(a, b))$ 的字典序排序或按 $a + b$ 的大小排序均可。

修复方案

正确策略：按 $(\min(a, b), \max(a, b))$ 的字典序排序或按 $a + b$ 的大小排序均可。

为什么有传递性？

修复方案

正确策略：按 $(\min(a, b), \max(a, b))$ 的字典序排序或按 $a + b$ 的大小排序均可。

为什么有传递性？

- 字典序和 $a + b$ 基于数字比大小，天然具有传递性；

修复方案

正确策略：按 $(\min(a, b), \max(a, b))$ 的字典序排序或按 $a + b$ 的大小排序均可。

为什么有传递性？

- 字典序和 $a + b$ 基于数字比大小，天然具有传递性；
- `sort` 能正常工作，排序结果就是全局最优。

正确性证明

那么问题来了，这个贪心策略正确性为什么是对的？

正确性证明

那么问题来了，这个贪心策略正确性为什么是对的？
说实话，这个证明相当困难。整个洛谷题解区很多题解都是语焉
不详一笔带过的，不懂怎么过的审核。

正确性证明

那么问题来了，这个贪心策略正确性为什么是对的？

说实话，这个证明相当困难。整个洛谷题解区很多题解都是语焉不详一笔带过的，不懂怎么过的审核。

作为做题人，有时候不一定要完整证明。事实上，赛场上遇到这种题，你完全可以观察样例猜出这个结论，提交，通过，然后就不管了。

正确性证明

那么问题来了，这个贪心策略正确性为什么是对的？

说实话，这个证明相当困难。整个洛谷题解区很多题解都是语焉不详一笔带过的，不懂怎么过的审核。

作为做题人，有时候不一定要完整证明。事实上，赛场上遇到这种题，你完全可以观察样例猜出这个结论，提交，通过，然后就不管了。

但是作为讲课者，这种不严谨证明完全是对听众的不负责任，因此我花了很多时间和很大精力证明，虽然最后还是没想到什么很优美的证法，但是功夫不负有心人，我用 AI 生成出了一个简要证明，我仔细看过，应该没什么大体问题。

正确性证明

设每个数组原始为 $a_i = (x_i, y_i)$ 。

内部逆序对贡献为 $[x_i > y_i]$ ，这个值是**固定不变的**，无论数组怎么排列。

总逆序对 = **内部逆序对之和** + **跨块逆序对之和**。

因此，最小化总逆序对等价于最小化跨块逆序对，而跨块逆序对**只与每个数组的两个数值有关**，与它们的内部顺序无关（因为跨块时，一个数组的两个元素都会与另一个数组的两个元素各比较一次，顺序不影响比较结果）。

所以，我们可以把每个数组视为一个**无序集合** $\{l_i, r_i\}$ ，其中

$$l_i = \min(x_i, y_i), \quad r_i = \max(x_i, y_i).$$

用 (l_i, r_i) 表示该集合（默认 $l_i \leq r_i$ ），排序时只关心跨块逆序对。

正确性证明

相邻交换判据 (核心引理)

设两个相邻块 $A = (l_1, r_1)$, $B = (l_2, r_2)$, 且 $l_1 \leq r_1$, $l_2 \leq r_2$ 。

当 A 在 B 前时, 跨块逆序对数为

$$f(A, B) = [l_1 > l_2] + [l_1 > r_2] + [r_1 > l_2] + [r_1 > r_2],$$

当 B 在 A 前时, 为

$$f(B, A) = [l_2 > l_1] + [l_2 > r_1] + [r_2 > l_1] + [r_2 > r_1].$$

我们需要证明: 若满足以下任一排序规则, 则 $f(A, B) \leq f(B, A)$:

- 规则1 (字典序): $(l_1, r_1) \leq_{\text{lex}} (l_2, r_2)$, 即 $l_1 < l_2$ 或 $(l_1 = l_2 \text{ 且 } r_1 \leq r_2)$;
- 规则2 (和): $l_1 + r_1 \leq l_2 + r_2$ 。

正确性证明

证明 (分三类讨论) :

① $l_1 < l_2$

此时必有 $l_1 \leq l_2$, 规则1成立。规则2可能成立也可能不成立, 但我们分别验证:

- 若 $r_1 \leq l_2$, 则 A 中两数均不大于 B 中任意数, 故 $f(A, B) = 0 \leq f(B, A)$ 。
- 若 $l_2 < r_1 \leq r_2$, 则 $f(A, B) = 1$, 而 $f(B, A) \geq 2$ (因为 $l_2 > l_1$ 和 $r_2 > l_1$ 都为真), 所以成立。
- 若 $r_1 > r_2$, 则直接代入得 $f(A, B) = 2$, 且 $f(B, A) = 2$ (此时 $l_2 > r_1$ 和 $r_2 > r_1$ 均为假), 相等。

因此, 当 $l_1 < l_2$ 时, 两种规则都保证 $f(A, B) \leq f(B, A)$ 。

正确性证明

② $l_1 = l_2$

此时规则1和规则2都等价于 $r_1 \leq r_2$ 。

在该条件下：

- $f(A, B) = [r_1 > l_1]$ (因为 $r_1 > r_2$ 为假, $l_1 > l_2$ 为假, $l_1 > r_2$ 为假) 。
- $f(B, A) = [l_2 > r_1] + [r_2 > l_1] + [r_2 > r_1]$, 其中 $l_2 = l_1$, 且 $r_1 \leq r_2$ 。

若 $r_1 = l_1$, 则 $f(A, B) = 0$; 若 $r_1 > l_1$, 则 $r_2 \geq r_1 > l_1$, 于是 $[r_2 > l_1] = 1$, 所以 $f(B, A) \geq 1 = f(A, B)$ 。成立。

正确性证明

$$\textcircled{3} l_1 > l_2$$

规则1不成立, 只考虑规则2。由 $l_1 + r_1 \leq l_2 + r_2$ 且 $l_1 > l_2$, 可得 $r_1 < r_2$ 。于是

$$l_2 < l_1 \leq r_1 < r_2.$$

代入得

$$f(A, B) = [l_1 > l_2] = 1 + [l_1 > r_2] = 0 + [r_1 > l_2] = 1 + [r_1 > r_2] = 0 = 2,$$

$$f(B, A) = [l_2 > l_1] = 0 + [l_2 > r_1] = 0 + [r_2 > l_1] = 1 + [r_2 > r_1] = 1 = 2,$$

相等。

综上所述, 两种排序规则均满足相邻交换不会增加跨块逆序对。

那为什么交换相邻的就够了？

等下，你是不是漏掉了什么……？

那为什么交换相邻的就够了？

等下，你是不是漏掉了什么……？

是的，我们还没有严谨说明分析相邻逆序对就能得到比较规则。

那为什么交换相邻的就够了？

等下，你是不是漏掉了什么……？

是的，我们还没有严谨说明分析相邻逆序对就能得到比较规则。

我们刚才证明了：交换相邻逆序对，答案不会变差。

那为什么交换相邻的就够了？

等下，你是不是漏掉了什么……？

是的，我们还没有严谨说明分析相邻逆序对就能得到比较规则。

我们刚才证明了：交换相邻逆序对，答案不会变差。

问题是：我们只比较了相邻的两个人，凭什么能保证任意两个人之间的顺序是对的？

那为什么交换相邻的就够了？

等下，你是不是漏掉了什么……？

是的，我们还没有严谨说明分析相邻逆序对就能得到比较规则。

我们刚才证明了：交换相邻逆序对，答案不会变差。

问题是：我们只比较了相邻的两个人，凭什么能保证任意两个人之间的顺序是对的？

想想昨天所学的冒泡排序。

冒泡排序与逆序对

```
1  for (int i = 0; i < len; ++i)
2      for (int j = len - 1; j > i; --j)
3          if (arr[j] < arr[j - 1])
4              swap (arr[j] , arr[j - 1]);
```

冒泡排序与逆序对

冒泡排序每次比较相邻两个元素，如果顺序不对就交换，直到没有相邻逆序对为止。

冒泡排序与逆序对

冒泡排序每次比较相邻两个元素，如果顺序不对就交换，直到没有相邻逆序对为止。

这个过程结束时，序列一定完全有序——只要比较函数满足传递性，这就是冒泡排序正确性的基本结论。

冒泡排序与逆序对

冒泡排序每次比较相邻两个元素，如果顺序不对就交换，直到没有相邻逆序对为止。

这个过程结束时，序列一定完全有序——只要比较函数满足传递性，这就是冒泡排序正确性的基本结论。

所以“只比较相邻的两个人”就够，因为通过反复交换相邻逆序对，任何乱序序列都能变成有序序列。

冒泡排序与逆序对

冒泡排序每次比较相邻两个元素，如果顺序不对就交换，直到没有相邻逆序对为止。

这个过程结束时，序列一定完全有序——只要比较函数满足传递性，这就是冒泡排序正确性的基本结论。

所以“只比较相邻的两个人”就够，因为通过反复交换相邻逆序对，任何乱序序列都能变成有序序列。

而我们前面证明了：交换相邻逆序对，答案不会变差。

冒泡排序与逆序对

冒泡排序每次比较相邻两个元素，如果顺序不对就交换，直到没有相邻逆序对为止。

这个过程结束时，序列一定完全有序——只要比较函数满足传递性，这就是冒泡排序正确性的基本结论。

所以“只比较相邻的两个人”就够，因为通过反复交换相邻逆序对，任何乱序序列都能变成有序序列。

而我们前面证明了：交换相邻逆序对，答案不会变差。

所以任意一个乱序序列，都可以通过不断交换相邻逆序对，变成一个有序序列，而且答案全程不增。

冒泡排序与逆序对

冒泡排序每次比较相邻两个元素，如果顺序不对就交换，直到没有相邻逆序对为止。

这个过程结束时，序列一定完全有序——只要比较函数满足传递性，这就是冒泡排序正确性的基本结论。

所以“只比较相邻的两个人”就够，因为通过反复交换相邻逆序对，**任何乱序序列都能变成有序序列**。

而我们前面证明了：交换相邻逆序对，答案不会变差。

所以任意一个乱序序列，都可以通过不断交换相邻逆序对，变成一个有序序列，而且**答案全程不增**。

因此这个有序序列的答案 $\leq$ 任意一个乱序序列的答案。

冒泡排序与逆序对

冒泡排序每次比较相邻两个元素，如果顺序不对就交换，直到没有相邻逆序对为止。

这个过程结束时，序列一定完全有序——只要比较函数满足传递性，这就是冒泡排序正确性的基本结论。

所以“只比较相邻的两个人”就够，因为通过反复交换相邻逆序对，**任何乱序序列都能变成有序序列**。

而我们前面证明了：交换相邻逆序对，答案不会变差。

所以任意一个乱序序列，都可以通过不断交换相邻逆序对，变成一个有序序列，而且**答案全程不增**。

因此这个有序序列的答案 $\leq$ 任意一个乱序序列的答案。

所以，**这个有序序列就是全局最优**。

总结

邻项交换的完整流程，现在有三步：

总结

邻项交换的完整流程，现在有三步：

- 1 **列式子**：考虑相邻两个人 i 和 j ，列出交换前后的总代价，作差，定出比较规则 `cmp`；

总结

邻项交换的完整流程，现在有三步：

- 1 **列式子**：考虑相邻两个人 i 和 j ，列出交换前后的总代价，作差，定出比较规则 `cmp`；
- 2 **验传递性**：检查 `cmp` 是否满足传递性——如果不满足，`sort` 会出问题，需要寻找一个等价的、有传递性的标量特征；

总结

邻项交换的完整流程，现在有三步：

- 1 **列式子**：考虑相邻两个人 i 和 j ，列出交换前后的总代价，作差，定出比较规则 `cmp`；
- 2 **验传递性**：检查 `cmp` 是否满足传递性——如果不满足，`sort` 会出问题，需要寻找一个等价的、有传递性的标量特征；
- 3 **排序**：用 `sort` 排序，得到全局最优。

总结

核心教训：

总结

核心教训：

- 列式子只是第一步；

总结

核心教训：

- 列式子只是第一步；
- 传递性是把局部结论推广到全局的桥梁；

总结

核心教训：

- 列式子只是第一步；
- 传递性是把局部结论推广到全局的桥梁；
- 如果传递性被破坏，整个贪心证明就崩塌了。

目录

- 1 引入
- 2 邻项交换
- 3 区间选点与区间覆盖
- 4 绝对值模型
- 5 哈夫曼模型
- 6 进阶技巧
- 7 总结

从排序比较到几何选择

前两部分我们一直在做同一件事：**给元素排序**。

从排序比较到几何选择

前两部分我们一直在做同一件事：**给元素排序**。
排队接水是给“人”排序，平衡者是给“预期分数”排序，
CF2023A是给“二元组”排序。

从排序比较到几何选择

前两部分我们一直在做同一件事：**给元素排序**。

排队接水是给“人”排序，平衡者是给“预期分数”排序，

CF2023A是给“二元组”排序。

排序的依据都是“谁应该排在谁前面”——这是邻项交换解决的问题。

从排序比较到几何选择

前两部分我们一直在做同一件事：**给元素排序**。

排队接水是给“人”排序，平衡者是给“预期分数”排序，

CF2023A是给“二元组”排序。

排序的依据都是“谁应该排在谁前面”——这是邻项交换解决的问题。

但从这一部分开始，问题的形态变了。

从排序比较到几何选择

前两部分我们一直在做同一件事：**给元素排序**。

排队接水是给“人”排序，平衡者是给“预期分数”排序，

CF2023A是给“二元组”排序。

排序的依据都是“谁应该排在谁前面”——这是邻项交换解决的问题。

但从这一部分开始，问题的形态变了。

不再是“给一堆东西排个序”，而是在数轴上选点、选线段。

从排序比较到几何选择

前两部分我们一直在做同一件事：**给元素排序**。

排队接水是给“人”排序，平衡者是给“预期分数”排序，

CF2023A是给“二元组”排序。

排序的依据都是“谁应该排在谁前面”——这是邻项交换解决的问题。

但从这一部分开始，问题的形态变了。

不再是“给一堆东西排个序”，而是在**数轴上选点、选线段**。

排序依然会出现，但排序的目的变了——不是为了排出一个“顺序”，而是为了**方便扫描**。

从排序比较到几何选择

前两部分我们一直在做同一件事：**给元素排序**。

排队接水是给“人”排序，平衡者是给“预期分数”排序，

CF2023A是给“二元组”排序。

排序的依据都是“谁应该排在谁前面”——这是邻项交换解决的问题。

但从这一部分开始，问题的形态变了。

不再是“给一堆东西排个序”，而是在**数轴上选点、选线段**。

排序依然会出现，但排序的目的变了——不是为了排出一个“顺序”，而是为了**方便扫描**。

这就是区间问题。

区间选点问题

题目描述

给定 N 个闭区间 $[L_i, R_i]$ ，在数轴上放尽量少的点，使每个区间至少包含一个点。

区间选点：先简化问题

拿到一堆区间，先别急着排序。先观察区间之间的关系。

区间选点：先简化问题

拿到一堆区间，先别急着排序。先观察区间之间的关系。
如果区间 A 完全包含区间 B （即 $L_A \leq L_B$ 且 $R_B \leq R_A$ ）：

区间选点：先简化问题

拿到一堆区间，先别急着排序。先观察区间之间的关系。

如果区间 A 完全包含区间 B （即 $L_A \leq L_B$ 且 $R_B \leq R_A$ ）：

如果 B 里面放了一个点，那么 A 肯定也被覆盖了（因为 B 在 A 里面）。

区间选点：先简化问题

拿到一堆区间，先别急着排序。先观察区间之间的关系。

如果区间 A 完全包含区间 B （即 $L_A \leq L_B$ 且 $R_B \leq R_A$ ）：

如果 B 里面放了一个点，那么 A 肯定也被覆盖了（因为 B 在 A 里面）。

反过来，如果只在 A 里面放点， B 不一定被覆盖。

区间选点：先简化问题

拿到一堆区间，先别急着排序。先观察区间之间的关系。

如果区间 A 完全包含区间 B （即 $L_A \leq L_B$ 且 $R_B \leq R_A$ ）：

如果 B 里面放了一个点，那么 A 肯定也被覆盖了（因为 B 在 A 里面）。

反过来，如果只在 A 里面放点， B 不一定被覆盖。

所以，大的区间 A 是“多余”的——覆盖了 B 就自动覆盖了 A ，我们完全不用管 A 。

区间选点：先简化问题

拿到一堆区间，先别急着排序。先观察区间之间的关系。

如果区间 A 完全包含区间 B （即 $L_A \leq L_B$ 且 $R_B \leq R_A$ ）：

如果 B 里面放了一个点，那么 A 肯定也被覆盖了（因为 B 在 A 里面）。

反过来，如果只在 A 里面放点， B 不一定被覆盖。

所以，大的区间 A 是“多余”的——覆盖了 B 就自动覆盖了 A ，我们完全不用管 A 。

因此第一步：删掉所有包含其它区间的区间，只保留那些“不被任何其它区间包含”的区间。

删完之后区间长什么样？

删掉被包含的区间之后，剩下的区间有一个非常好的性质：

删完之后区间长什么样？

删掉被包含的区间之后，剩下的区间有一个非常好的性质：
左端点和右端点同时递增。

删完之后区间长什么样？

删掉被包含的区间之后，剩下的区间有一个非常好的性质：

左端点和右端点同时递增。

也就是说，把它们按左端点从小到大排好之后，右端点也是从小到大排好的：

$$L_1 < L_2 < \cdots < L_k, \quad R_1 < R_2 < \cdots < R_k$$

删完之后区间长什么样？

删掉被包含的区间之后，剩下的区间有一个非常好的性质：

左端点和右端点同时递增。

也就是说，把它们按左端点从小到大排好之后，右端点也是从小到大排好的：

$$L_1 < L_2 < \cdots < L_k, \quad R_1 < R_2 < \cdots < R_k$$

为什么？如果 $L_1 \leq L_2$ 但 $R_1 \geq R_2$ ，那就说明区间 1 包含了区间 2，应该被删掉——矛盾。

删完之后区间长什么样？

删掉被包含的区间之后，剩下的区间有一个非常好的性质：

左端点和右端点同时递增。

也就是说，把它们按左端点从小到大排好之后，右端点也是从小到大排好的：

$$L_1 < L_2 < \cdots < L_k, \quad R_1 < R_2 < \cdots < R_k$$

为什么？如果 $L_1 \leq L_2$ 但 $R_1 \geq R_2$ ，那就说明区间 1 包含了区间 2，应该被删掉——矛盾。

所以删完之后，区间就像一排“阶梯”一样，从左往右依次延伸。

最左边的区间怎么办？

区间 $[L_1, R_1]$ 是所有区间中左端点最小、右端点也最小的区间。

最左边的区间怎么办？

区间 $[L_1, R_1]$ 是所有区间中左端点最小、右端点也最小的区间。
为了覆盖它，我们必须在 $[L_1, R_1]$ 内部放一个点。

最左边的区间怎么办？

区间 $[L_1, R_1]$ 是所有区间中左端点最小、右端点也最小的区间。
为了覆盖它，我们必须在 $[L_1, R_1]$ 内部放一个点。
这个点放在哪里最好？

最左边的区间怎么办？

区间 $[L_1, R_1]$ 是所有区间中左端点最小、右端点也最小的区间。

为了覆盖它，我们必须在 $[L_1, R_1]$ 内部放一个点。

这个点放在哪里最好？

如果放在 L_1 附近，它可能覆盖不到后面的区间；

最左边的区间怎么办？

区间 $[L_1, R_1]$ 是所有区间中左端点最小、右端点也最小的区间。

为了覆盖它，我们必须在 $[L_1, R_1]$ 内部放一个点。

这个点放在哪里最好？

如果放在 L_1 附近，它可能覆盖不到后面的区间；

如果放在 R_1 ，它是 $[L_1, R_1]$ 内最靠右的选择，能覆盖到尽可能多的后续区间。

最左边的区间怎么办？

区间 $[L_1, R_1]$ 是所有区间中左端点最小、右端点也最小的区间。
为了覆盖它，我们必须在 $[L_1, R_1]$ 内部放一个点。
这个点放在哪里最好？

如果放在 L_1 附近，它可能覆盖不到后面的区间；

如果放在 R_1 ，它是 $[L_1, R_1]$ 内最靠右的选择，能覆盖到尽可能多的后续区间。

所以，在 R_1 放一个点，一定不差于放在 $[L_1, R_1]$ 内的任何其它位置。

最左边的区间怎么办？

区间 $[L_1, R_1]$ 是所有区间中左端点最小、右端点也最小的区间。
为了覆盖它，我们必须在 $[L_1, R_1]$ 内部放一个点。

这个点放在哪里最好？

如果放在 L_1 附近，它可能覆盖不到后面的区间；

如果放在 R_1 ，它是 $[L_1, R_1]$ 内最靠右的选择，能覆盖到尽可能多的后续区间。

所以，在 R_1 放一个点，一定不差于放在 $[L_1, R_1]$ 内的任何其它位置。

放了之后，所有左端点 $\leq R_1$ 的区间都被这个点覆盖了，删掉它们。

最左边的区间怎么办？

区间 $[L_1, R_1]$ 是所有区间中左端点最小、右端点也最小的区间。
为了覆盖它，我们必须在 $[L_1, R_1]$ 内部放一个点。

这个点放在哪里最好？

如果放在 L_1 附近，它可能覆盖不到后面的区间；

如果放在 R_1 ，它是 $[L_1, R_1]$ 内最靠右的选择，能覆盖到尽可能多的后续区间。

所以，在 R_1 放一个点，一定不差于放在 $[L_1, R_1]$ 内的任何其它位置。

放了之后，所有左端点 $\leq R_1$ 的区间都被这个点覆盖了，删掉它们。

剩下的区间还是同样形态——继续在最小的右端点放点。

策略自然形成

经过上面的分析，策略已经水到渠成了：

策略自然形成

经过上面的分析，策略已经水到渠成了：

- 1 删掉所有被包含的区间（它们不影响答案）；

策略自然形成

经过上面的分析，策略已经水到渠成了：

- 1 删掉所有被包含的区间（它们不影响答案）；
- 2 剩下的区间里，右端点最小的一定要先处理，在它的右端点放点；

策略自然形成

经过上面的分析，策略已经水到渠成了：

- 1 删掉所有被包含的区间（它们不影响答案）；
- 2 剩下的区间里，右端点最小的一定要先处理，在它的右端点放点；
- 3 删掉所有被这个点覆盖的区间；

策略自然形成

经过上面的分析，策略已经水到渠成了：

- 1 删掉所有被包含的区间（它们不影响答案）；
- 2 剩下的区间里，右端点最小的一定要先处理，在它的右端点放点；
- 3 删掉所有被这个点覆盖的区间；
- 4 重复直到所有区间都被覆盖。

具体实现

实现的时候不需要真的删区间——按右端点排序，扫描一遍即可。

具体实现

实现的时候不需要真的删区间——按右端点排序，扫描一遍即可。

这就是贪心策略的完整推导过程：不是凭空“按右端点排序”，而是从区间形态出发，一步步分析出来的。

区间覆盖

刚才的问题是“放尽量少的点去覆盖所有区间”。

区间覆盖

刚才的问题是“放尽量少的点去覆盖所有区间”。
现在反过来：用尽量少的给定区间，去完全覆盖一个大区间 $[L, R]$ 。

区间覆盖

刚才的问题是“放尽量少的点去覆盖所有区间”。

现在反过来：用尽量少的给定区间，去完全覆盖一个大区间 $[L, R]$ 。

这两个问题看起来相似，但贪心策略完全不同。

区间覆盖

刚才的问题是“放尽量少的点去覆盖所有区间”。

现在反过来：用尽量少的给定区间，去完全覆盖一个大区间 $[L, R]$ 。

这两个问题看起来相似，但贪心策略完全不同。

题目描述

给定目标区间 $[L, R]$ 和 N 个区间 $[L_i, R_i]$ ，选最少的区间覆盖 $[L, R]$ 。

区间覆盖：先找第一个区间

第一步：选哪个区间作为覆盖的开始？

区间覆盖：先找第一个区间

第一步：选哪个区间作为覆盖的开始？

覆盖的起点是 L ，所以第一个区间的左端点必须 $\leq L$ 。

区间覆盖：先找第一个区间

第一步：选哪个区间作为覆盖的开始？

覆盖的起点是 L ，所以第一个区间的左端点必须 $\leq L$ 。

在所有满足 $L_i \leq L$ 的区间中，选哪一个最好？

区间覆盖：先找第一个区间

第一步：选哪个区间作为覆盖的开始？

覆盖的起点是 L ，所以第一个区间的左端点必须 $\leq L$ 。

在所有满足 $L_i \leq L$ 的区间中，选哪一个最好？

直觉：选右端点最远的那个。因为它覆盖得最远，后面需要覆盖的部分最少。

区间覆盖：先找第一个区间

第一步：选哪个区间作为覆盖的开始？

覆盖的起点是 L ，所以第一个区间的左端点必须 $\leq L$ 。

在所有满足 $L_i \leq L$ 的区间中，选哪一个最好？

直觉：**选右端点最远的那个**。因为它覆盖得最远，后面需要覆盖的部分最少。

选了之后，当前覆盖到的位置变成了这个区间的右端点。

区间覆盖：先找第一个区间

第一步：选哪个区间作为覆盖的开始？

覆盖的起点是 L ，所以第一个区间的左端点必须 $\leq L$ 。

在所有满足 $L_i \leq L$ 的区间中，选哪一个最好？

直觉：**选右端点最远的那个**。因为它覆盖得最远，后面需要覆盖的部分最少。

选了之后，当前覆盖到的位置变成了这个区间的右端点。

然后重复：在所有左端点 $\leq$ 当前覆盖位置的区间中，选右端点最远的。

区间覆盖：为什么选最远右端点是对的？

假设当前覆盖到位置 cur ，所有 $L_i \leq \text{cur}$ 的区间里， $R_{\max}$ 是最大的右端点。

区间覆盖：为什么选最远右端点是对的？

假设当前覆盖到位置 cur ，所有 $L_i \leq \text{cur}$ 的区间里， $R_{\max}$ 是最大的右端点。

如果最优解在这里选了一个右端点 $r < R_{\max}$ 的区间：

区间覆盖：为什么选最远右端点是对的？

假设当前覆盖到位置 cur ，所有 $L_i \leq cur$ 的区间里， $R_{\max}$ 是最大的右端点。

如果最优解在这里选了一个右端点 $r < R_{\max}$ 的区间：
把它替换成 $R_{\max}$ 那个区间：

区间覆盖：为什么选最远右端点是对的？

假设当前覆盖到位置 cur ，所有 $L_i \leq \text{cur}$ 的区间里， $R_{\max}$ 是最大的右端点。

如果最优解在这里选了一个右端点 $r < R_{\max}$ 的区间：
把它替换成 $R_{\max}$ 那个区间：

- 新区间的左端点 $\leq \text{cur}$ ，所以能接上；

区间覆盖：为什么选最远右端点是对的？

假设当前覆盖到位置 cur ，所有 $L_i \leq cur$ 的区间里， $R_{\max}$ 是最大的右端点。

如果最优解在这里选了一个右端点 $r < R_{\max}$ 的区间：
把它替换成 $R_{\max}$ 那个区间：

- 新区间的左端点 $\leq cur$ ，所以能接上；
- 新区间的右端点更远，覆盖范围只大不小；

区间覆盖：为什么选最远右端点是对的？

假设当前覆盖到位置 cur ，所有 $L_i \leq cur$ 的区间里， $R_{\max}$ 是最大的右端点。

如果最优解在这里选了一个右端点 $r < R_{\max}$ 的区间：
把它替换成 $R_{\max}$ 那个区间：

- 新区间的左端点 $\leq cur$ ，所以能接上；
- 新区间的右端点更远，覆盖范围只大不小；
- 后续需要覆盖的部分变小了，选的区间数不可能增多。

区间覆盖：为什么选最远右端点是对的？

假设当前覆盖到位置 cur ，所有 $L_i \leq cur$ 的区间里， $R_{\max}$ 是最大的右端点。

如果最优解在这里选了一个右端点 $r < R_{\max}$ 的区间：
把它替换成 $R_{\max}$ 那个区间：

- 新区间的左端点 $\leq cur$ ，所以能接上；
- 新区间的右端点更远，覆盖范围只大不小；
- 后续需要覆盖的部分变小了，选的区间数不可能增多。
- 所以，选右端点最远的区间一定不劣。

具体实现

实现时按左端点排序，扫描一遍，每次取可选区间中右端点最大的即可。

两种区间问题的对比

	区间选点	区间覆盖
目标	点覆盖所有区间	区间覆盖一个大区间
排序方式	按右端点排序	按左端点排序
贪心决策	选右端点	选右端点最远的区间

两种区间问题的对比

	区间选点	区间覆盖
目标	点覆盖所有区间	区间覆盖一个大区间
排序方式	按右端点排序	按左端点排序
贪心决策	选右端点	选右端点最远的区间

共同点：排序都是为扫描服务的工具。

两种区间问题的对比

	区间选点	区间覆盖
目标	点覆盖所有区间	区间覆盖一个大区间
排序方式	按右端点排序	按左端点排序
贪心决策	选右端点	选右端点最远的区间

共同点：排序都是为扫描服务的工具。

不同点：排序的关键字不同，贪心的决策方式也不同。

区间动态变式——鹅鸭杀

前面的区间问题都是静态的：区间在那里不动，我们去选点或选区间。

区间动态变式——鹅鸭杀

前面的区间问题都是静态的：区间在那里不动，我们去选点或选区间。

但有些问题的区间是动态出现的。

区间动态变式——鹅鸭杀

前面的区间问题都是静态的：区间在那里不动，我们去选点或选区间。

但有些问题的区间是动态出现的。

题目描述（GYM105184C）

N 个人，第 i 人在当前人数 $K \in [L_i, R_i]$ 时愿意加入。你可以安排加入顺序，求最多能加入多少人。

问题分析

当前人数 K 从 0 开始不断增长。

问题分析

当前人数 K 从 0 开始不断增长。
每个人的有效窗口是 $[L_i, R_i]$:

问题分析

当前人数 K 从 0 开始不断增长。

每个人的有效窗口是 $[L_i, R_i]$:

- $K < L_i$: 还没到 ta 愿意加入的时候;

问题分析

当前人数 K 从 0 开始不断增长。

每个人的有效窗口是 $[L_i, R_i]$:

- $K < L_i$: 还没到 ta 愿意加入的时候;
- $L_i \leq K \leq R_i$: ta 愿意加入;

问题分析

当前人数 K 从 0 开始不断增长。

每个人的有效窗口是 $[L_i, R_i]$:

- $K < L_i$: 还没到 ta 愿意加入的时候;
- $L_i \leq K \leq R_i$: ta 愿意加入;
- $K > R_i$: ta 已经过期, 不会加入了。

贪心策略

在任意时刻 K , 所有 $L_i \leq K$ 的人都已经“可用”。

贪心策略

在任意时刻 K ，所有 $L_i \leq K$ 的人都已经“可用”。
其中有些人快要过期了（ R_i 很小），有些人还能等很久（ R_i 很大）。

贪心策略

在任意时刻 K ，所有 $L_i \leq K$ 的人都已经“可用”。
其中有些人快要过期了（ R_i 很小），有些人还能等很久（ R_i 很大）。
应该优先让谁加入？

贪心策略

在任意时刻 K ，所有 $L_i \leq K$ 的人都已经“可用”。

其中有些人快要过期了 (R_i 很小)，有些人还能等很久 (R_i 很大)。

应该优先让谁加入？

让 R_i 最小的人先加入。

贪心策略

在任意时刻 K , 所有 $L_i \leq K$ 的人都已经“可用”。

其中有些人快要过期了 (R_i 很小), 有些人还能等很久 (R_i 很大)。

应该优先让谁加入?

让 R_i 最小的人先加入。

因为 ta 最紧迫, 再不加入就过期了。 R_i 大的人还能再等等。

贪心策略

在任意时刻 K , 所有 $L_i \leq K$ 的人都已经“可用”。

其中有些人快要过期了 (R_i 很小), 有些人还能等很久 (R_i 很大)。

应该优先让谁加入?

让 R_i 最小的人先加入。

因为 ta 最紧迫, 再不加入就过期了。 R_i 大的人还能再等等。

这就是贪心策略: 每次从所有可用的人中, 选 R_i 最小的加入, 然后 $K \leftarrow K + 1$, 继续这个过程。

贪心策略

在任意时刻 K , 所有 $L_i \leq K$ 的人都已经“可用”。

其中有些人快要过期了 (R_i 很小), 有些人还能等很久 (R_i 很大)。

应该优先让谁加入?

让 R_i 最小的人先加入。

因为 ta 最紧迫, 再不加入就过期了。 R_i 大的人还能再等等。

这就是贪心策略: 每次从所有可用的人中, 选 R_i 最小的加入, 然后 $K \leftarrow K + 1$, 继续这个过程。

至于怎么快速找到 R_i 最小的可用人——那是实现层面的问题, 用合适的数据结构维护即可, 但贪心策略本身已经确定下来了。

第三部分小结

区间问题教会了我们三件事：

第三部分小结

区间问题教会了我们三件事：

- 1 贪心的对象不一定是“排序”。区间选点、区间覆盖，都是在几何空间里做选择。

第三部分小结

区间问题教会了我们三件事：

- 1 贪心的对象不一定是“排序”。区间选点、区间覆盖，都是在几何空间里做选择。
- 2 排序的目的变了。之前排序是为了排出一个“顺序”，区间问题里排序是为了“方便扫描”。

第三部分小结

区间问题教会了我们三件事：

- 1 贪心的对象不一定是“排序”。区间选点、区间覆盖，都是在几何空间里做选择。
- 2 排序的目的变了。之前排序是为了排出一个“顺序”，区间问题里排序是为了“方便扫描”。
- 3 策略要从问题结构出发。区间选点要先删被包含区间，区间覆盖要按左端点扫描选最远右端——这些策略不是凭空猜的，是从区间形态分析出来的。

目录

- 1 引入
- 2 邻项交换
- 3 区间选点与区间覆盖
- 4 绝对值模型
- 5 哈夫曼模型
- 6 进阶技巧
- 7 总结

从覆盖到距离

前面我们处理的问题都是“选若干个东西”：

从覆盖到距离

前面我们处理的问题都是“选若干个东西”：

- 区间选点：选若干个点；

从覆盖到距离

前面我们处理的问题都是“选若干个东西”：

- 区间选点：选若干个点；
- 区间覆盖：选若干个区间；

从覆盖到距离

前面我们处理的问题都是“选若干个东西”：

- 区间选点：选若干个点；
- 区间覆盖：选若干个区间；
- 鹅鸭杀：选若干个人。

选点问题

现在换一个角度：只选一个位置，让所有点到它的距离之和最小。

选点问题

现在换一个角度：只选一个位置，让所有点到它的距离之和最小。

P10452 货仓选址

数轴上有 N 个点 A_i ，选一个位置 X ，使 $\sum |A_i - X|$ 最小。

配对消去法

这个问题在数学上有个经典结论： X 取中位数。

配对消去法

这个问题在数学上有个经典结论： X 取中位数。
但我们不只需要知道结论，还需要知道为什么。
先把 A 排序： $A_1 \leq A_2 \leq \cdots \leq A_N$ 。

配对消去法

这个问题在数学上有个经典结论： X 取中位数。

但我们不只需要知道结论，还需要知道为什么。

先把 A 排序： $A_1 \leq A_2 \leq \cdots \leq A_N$ 。

考虑最左边和最右边的两个点： A_1 和 A_N 。

配对消去法

这个问题在数学上有个经典结论： X 取中位数。

但我们不只需要知道结论，还需要知道为什么。

先把 A 排序： $A_1 \leq A_2 \leq \cdots \leq A_N$ 。

考虑最左边和最右边的两个点： A_1 和 A_N 。

对于任意 X ，有：

$$|A_1 - X| + |A_N - X| \geq A_N - A_1$$

配对消去法

这个问题在数学上有个经典结论： X 取中位数。

但我们不只需要知道结论，还需要知道为什么。

先把 A 排序： $A_1 \leq A_2 \leq \cdots \leq A_N$ 。

考虑最左边和最右边的两个点： A_1 和 A_N 。

对于任意 X ，有：

$$|A_1 - X| + |A_N - X| \geq A_N - A_1$$

等号什么时候成立？当 X 落在 $[A_1, A_N]$ 之间时。

配对消去法

这个问题在数学上有个经典结论： X 取中位数。

但我们不只需要知道结论，还需要知道为什么。

先把 A 排序： $A_1 \leq A_2 \leq \cdots \leq A_N$ 。

考虑最左边和最右边的两个点： A_1 和 A_N 。

对于任意 X ，有：

$$|A_1 - X| + |A_N - X| \geq A_N - A_1$$

等号什么时候成立？当 X 落在 $[A_1, A_N]$ 之间时。

也就是说，这两个点的距离贡献，至少是 $A_N - A_1$ ，而且只要 X 在它们中间就能取到。

配对消去法

这个问题在数学上有个经典结论： X 取中位数。

但我们不只需要知道结论，还需要知道为什么。

先把 A 排序： $A_1 \leq A_2 \leq \cdots \leq A_N$ 。

考虑最左边和最右边的两个点： A_1 和 A_N 。

对于任意 X ，有：

$$|A_1 - X| + |A_N - X| \geq A_N - A_1$$

等号什么时候成立？当 X 落在 $[A_1, A_N]$ 之间时。

也就是说，这两个点的距离贡献，至少是 $A_N - A_1$ ，而且只要 X 在它们中间就能取到。

接下来看 A_2 和 A_{N-1} ：

$$|A_2 - X| + |A_{N-1} - X| \geq A_{N-1} - A_2$$

配对消去法

这个问题在数学上有个经典结论： X 取中位数。

但我们不只需要知道结论，还需要知道为什么。

先把 A 排序： $A_1 \leq A_2 \leq \cdots \leq A_N$ 。

考虑最左边和最右边的两个点： A_1 和 A_N 。

对于任意 X ，有：

$$|A_1 - X| + |A_N - X| \geq A_N - A_1$$

等号什么时候成立？当 X 落在 $[A_1, A_N]$ 之间时。

也就是说，这两个点的距离贡献，至少是 $A_N - A_1$ ，而且只要 X 在它们中间就能取到。

接下来看 A_2 和 A_{N-1} ：

$$|A_2 - X| + |A_{N-1} - X| \geq A_{N-1} - A_2$$

以此类推，一层一层“剥洋葱”。

配对消去法

把所有不等式加起来：

$$\sum_{i=1}^N |A_i - X| \geq (A_N - A_1) + (A_{N-1} - A_2) + \dots$$

配对消去法

把所有不等式加起来：

$$\sum_{i=1}^N |A_i - X| \geq (A_N - A_1) + (A_{N-1} - A_2) + \dots$$

右边是一个**常数**，跟 X 无关。

配对消去法

把所有不等式加起来：

$$\sum_{i=1}^N |A_i - X| \geq (A_N - A_1) + (A_{N-1} - A_2) + \dots$$

右边是一个**常数**，跟 X 无关。

等号成立的条件是： X 同时落在每一对配对的区间里。

配对消去法

把所有不等式加起来：

$$\sum_{i=1}^N |A_i - X| \geq (A_N - A_1) + (A_{N-1} - A_2) + \dots$$

右边是一个**常数**，跟 X 无关。

等号成立的条件是： X 同时落在每一对配对的区间里。

这些区间的交集是哪里？

配对消去法

把所有不等式加起来：

$$\sum_{i=1}^N |A_i - X| \geq (A_N - A_1) + (A_{N-1} - A_2) + \dots$$

右边是一个**常数**，跟 X 无关。

等号成立的条件是： X 同时落在每一对配对的区间里。

这些区间的交集是哪里？

- 如果 N 是奇数：最中间的那个点—— $A_{N/2}$ （下标从 0 开始）；

配对消去法

把所有不等式加起来：

$$\sum_{i=1}^N |A_i - X| \geq (A_N - A_1) + (A_{N-1} - A_2) + \dots$$

右边是一个**常数**，跟 X 无关。

等号成立的条件是： X 同时落在每一对配对的区间里。

这些区间的交集是哪里？

- 如果 N 是奇数：最中间的那个点—— $A_{N/2}$ （下标从 0 开始）；
- 如果 N 是偶数：最中间两个点之间的任意位置，代码取 $A_{N/2}$ 即可。

配对消去法

把所有不等式加起来：

$$\sum_{i=1}^N |A_i - X| \geq (A_N - A_1) + (A_{N-1} - A_2) + \dots$$

右边是一个**常数**，跟 X 无关。

等号成立的条件是： X 同时落在每一对配对的区间里。

这些区间的交集是哪里？

- 如果 N 是奇数：最中间的那个点—— $A_{N/2}$ （下标从 0 开始）；
- 如果 N 是偶数：最中间两个点之间的任意位置，代码取 $A_{N/2}$ 即可。
- 所以答案就是：**排序后取中位数**。

第四部分小结

绝对值模型和前面的区间模型都在数轴上，但思考方式完全不同：

第四部分小结

绝对值模型和前面的区间模型都在数轴上，但思考方式完全不同：

- 区间问题：选若干个东西去“覆盖”或“命中”；

第四部分小结

绝对值模型和前面的区间模型都在数轴上，但思考方式完全不同：

- 区间问题：选若干个东西去“覆盖”或“命中”；
- 绝对值问题：选一个位置去“接近”所有点。

第四部分小结

证明方法也不同：

第四部分小结

证明方法也不同：

- 区间问题用“交换论证”或“替换论证”；

第四部分小结

证明方法也不同：

- 区间问题用“交换论证”或“替换论证”；
- 绝对值问题用“配对消去法”——把最左和最右配对，一层层剥掉。

目录

- 1 引入
- 2 邻项交换
- 3 区间选点与区间覆盖
- 4 绝对值模型
- 5 哈夫曼模型
- 6 进阶技巧
- 7 总结

当数据不再静态

到目前为止，所有问题都有一个共同点：**数据是静态的。**

当数据不再静态

到目前为止，所有问题都有一个共同点：**数据是静态的**。
排队接水：人就在那里，排个序就完了。

当数据不再静态

到目前为止，所有问题都有一个共同点：**数据是静态的。**

排队接水：人就在那里，排个序就完了。

区间问题：区间就在那里，排个序扫一遍。

当数据不再静态

到目前为止，所有问题都有一个共同点：**数据是静态的。**

排队接水：人就在那里，排个序就完了。

区间问题：区间就在那里，排个序扫一遍。

绝对值：点就在那里，排个序取中位数。

当数据不再静态

到目前为止，所有问题都有一个共同点：**数据是静态的。**

排队接水：人就在那里，排个序就完了。

区间问题：区间就在那里，排个序扫一遍。

绝对值：点就在那里，排个序取中位数。

但如果每次操作会**产生新的数据**呢？

当数据不再静态

到目前为止，所有问题都有一个共同点：**数据是静态的。**

排队接水：人就在那里，排个序就完了。

区间问题：区间就在那里，排个序扫一遍。

绝对值：点就在那里，排个序取中位数。

但如果每次操作会**产生新的数据**呢？

P1090 合并果子

N 堆果子，重量 A_i 。每次合并两堆，代价为两堆重量之和。求合并成一堆的最小总代价。

合并果子：贪心策略

直觉：每次选最小的两堆合并。

合并果子：贪心策略

直觉：每次选最小的两堆合并。
为什么？

合并果子：贪心策略

直觉：每次选最小的两堆合并。

为什么？

考虑整个合并过程，它对应一棵二叉树：

合并果子：贪心策略

直觉：每次选最小的两堆合并。

为什么？

考虑整个合并过程，它对应一棵二叉树：

- 叶子是初始的 N 堆果子；

合并果子：贪心策略

直觉：每次选最小的两堆合并。

为什么？

考虑整个合并过程，它对应一棵二叉树：

- 叶子是初始的 N 堆果子；
- 内部节点是每次合并产生的新堆；

合并果子：贪心策略

直觉：每次选最小的两堆合并。

为什么？

考虑整个合并过程，它对应一棵二叉树：

- 叶子是初始的 N 堆果子；
- 内部节点是每次合并产生的新堆；
- 一个果子被合并的次数 = 它在树中的深度。

合并果子：贪心策略

$$\text{总代价} = \sum A_i \times \text{depth}_i。$$

合并果子：贪心策略

总代价 = $\sum A_i \times \text{depth}_i$ 。

为了让总代价最小，重量大的果子应该尽量浅（少合并），重量小的果子应该尽量深（多合并）。

合并果子：贪心策略

总代价 = $\sum A_i \times \text{depth}_i$ 。

为了让总代价最小，**重量大的果子应该尽量浅（少合并），重量小的果子应该尽量深（多合并）。**

所以每次选最小的两堆合并——让最小的果子最早被合并，也就是放在最深层。

合并果子：贪心策略

总代价 = $\sum A_i \times \text{depth}_i$ 。

为了让总代价最小，**重量大的果子应该尽量浅（少合并），重量小的果子应该尽量深（多合并）。**

所以每次选最小的两堆合并——让最小的果子最早被合并，也就是放在最深层。

至于怎么快速找到当前最小的两堆——那是实现层面的问题，用合适的数据结构维护即可，贪心策略本身已经确定下来了。

第五部分小结

合并果子标志着一次思维跃迁：

第五部分小结

合并果子标志着一次思维跃迁：

- 之前：排序一次，从头取到尾；

第五部分小结

合并果子标志着一次思维跃迁：

- 之前：排序一次，从头取到尾；
- 现在：每次取完产生新数据，需要动态维护。

第五部分小结

但贪心的本质没变：每次取当前最优。

第五部分小结

但贪心的本质没变：每次取当前最优。
变的只是怎么快速拿到当前最优：

第五部分小结

但贪心的本质没变：每次取当前最优。
变的只是怎么快速拿到当前最优：

- 静态数据 → 排序一次，依次取；

第五部分小结

但贪心的本质没变：每次取当前最优。

变的只是怎么快速拿到当前最优：

- 静态数据 → 排序一次，依次取；
- 动态数据 → 每次操作后用合适的数据结构维护。

目录

- 1 引入
- 2 邻项交换
- 3 区间选点与区间覆盖
- 4 绝对值模型
- 5 哈夫曼模型
- 6 进阶技巧
- 7 总结

进入进阶技巧

前面五部分覆盖了较为常见与简单的贪心模型：

进入进阶技巧

前面五部分覆盖了较为常见与简单的贪心模型：

- 买物品、选物品（排序取最值）；

进入进阶技巧

前面五部分覆盖了较为常见与简单的贪心模型：

- 买物品、选物品（排序取最值）；
- 邻项交换（排序定规则）；

进入进阶技巧

前面五部分覆盖了较为常见与简单的贪心模型：

- 买物品、选物品（排序取最值）；
- 邻项交换（排序定规则）；
- 区间选点、区间覆盖（排序扫描）；

进入进阶技巧

前面五部分覆盖了较为常见与简单的贪心模型：

- 买物品、选物品（排序取最值）；
- 邻项交换（排序定规则）；
- 区间选点、区间覆盖（排序扫描）；
- 绝对值模型（排序取中位数）；

进入进阶技巧

前面五部分覆盖了较为常见与简单的贪心模型：

- 买物品、选物品（排序取最值）；
- 邻项交换（排序定规则）；
- 区间选点、区间覆盖（排序扫描）；
- 绝对值模型（排序取中位数）；
- 哈夫曼模型（动态维护最值）。

进入进阶技巧

接下来的内容面向难度较高，属于**进阶技巧**。

进入进阶技巧

接下来的内容面向难度较高，属于**进阶技巧**。
它们不是全新的模型，而是在已有模型基础上的**思维拓展**。

树上贪心 I：父亲与儿子合并

UVA1205 Color a Tree

给定一棵有根树，每个节点有权值。要给所有节点染色，父节点必须在其子节点之前染色。节点染色的代价 = 时间戳 $\times$ 权值。求最小总代价。

这题如果用树形 DP 做，状态极其复杂。

这题如果用树形 DP 做，状态极其复杂。
但有一个漂亮的贪心结论：

这题如果用树形 DP 做，状态极其复杂。

但有一个漂亮的贪心结论：

当前“等效权值”最大的节点，一定紧随其父亲之后被染色。

这题如果用树形 DP 做，状态极其复杂。

但有一个漂亮的贪心结论：

当前“等效权值”最大的节点，一定紧随其父亲之后被染色。

其中“等效权值” = 块内权值和 / 块内点数。

这题如果用树形 DP 做，状态极其复杂。

但有一个漂亮的贪心结论：

当前“等效权值”最大的节点，一定紧随其父亲之后被染色。

其中“等效权值” = 块内权值和 / 块内点数。

所以做法是：

这题如果用树形 DP 做，状态极其复杂。

但有一个漂亮的贪心结论：

当前“等效权值”最大的节点，一定紧随其父亲之后被染色。

其中“等效权值” = 块内权值和 / 块内点数。

所以做法是：

- 1 找出等效权值最大的非根节点 u ;

这题如果用树形 DP 做，状态极其复杂。

但有一个漂亮的贪心结论：

当前“等效权值”最大的节点，一定紧随其父亲之后被染色。

其中“等效权值” = 块内权值和 / 块内点数。

所以做法是：

- 1 找出等效权值最大的非根节点 u ；
- 2 把 u 合并到它的父亲 fa 上；

这题如果用树形 DP 做，状态极其复杂。

但有一个漂亮的贪心结论：

当前“等效权值”最大的节点，一定紧随其父亲之后被染色。

其中“等效权值” = 块内权值和 / 块内点数。

所以做法是：

- 1 找出等效权值最大的非根节点 u ；
- 2 把 u 合并到它的父亲 fa 上；
- 3 更新 fa 的等效权值（权值和、点数都合并）；

这题如果用树形 DP 做，状态极其复杂。

但有一个漂亮的贪心结论：

当前“等效权值”最大的节点，一定紧随其父亲之后被染色。

其中“等效权值” = 块内权值和 / 块内点数。

所以做法是：

- 1 找出等效权值最大的非根节点 u ；
- 2 把 u 合并到它的父亲 fa 上；
- 3 更新 fa 的等效权值（权值和、点数都合并）；
- 4 用堆维护最大值，用并查集维护父亲关系。

树上贪心 II: 从叶子往上

最小支配集

给定一棵树，选择最少的点，使得每个点要么自己被选，要么其邻居被选。

另一个常见的树上贪心策略：从叶子往上考虑。

另一个常见的树上贪心策略：从叶子往上考虑。
为什么从叶子开始？

另一个常见的树上贪心策略：**从叶子往上考虑。**

为什么从叶子开始？

因为叶子只有一个邻居（它的父亲），自由度最低，最容易“被迫做出决定”。

另一个常见的树上贪心策略：**从叶子往上考虑。**

为什么从叶子开始？

因为叶子只有一个邻居（它的父亲），自由度最低，最容易“被迫做出决定”。

这就是“叶子剥除”的思想——每次处理自由度最低的节点。

另一个常见的树上贪心策略：**从叶子往上考虑。**

为什么从叶子开始？

因为叶子只有一个邻居（它的父亲），自由度最低，最容易“被迫做出决定”。

这就是“叶子剥除”的思想——每次处理自由度最低的节点。

策略：

另一个常见的树上贪心策略：从叶子往上考虑。

为什么从叶子开始？

因为叶子只有一个邻居（它的父亲），自由度最低，最容易“被迫做出决定”。

这就是“叶子剥除”的思想——每次处理自由度最低的节点。

策略：

- 1 找到一个叶子 u ;

另一个常见的树上贪心策略：从叶子往上考虑。

为什么从叶子开始？

因为叶子只有一个邻居（它的父亲），自由度最低，最容易“被迫做出决定”。

这就是“叶子剥除”的思想——每次处理自由度最低的节点。

策略：

- 1 找到一个叶子 u ;
- 2 如果 u 还没被覆盖，选它的父亲 v 一定不劣于选 u 自己;

另一个常见的树上贪心策略：从叶子往上考虑。

为什么从叶子开始？

因为叶子只有一个邻居（它的父亲），自由度最低，最容易“被迫做出决定”。

这就是“叶子剥除”的思想——每次处理自由度最低的节点。

策略：

- 1 找到一个叶子 u ;
- 2 如果 u 还没被覆盖，选它的父亲 v 一定不劣于选 u 自己;
- 3 因为 v 覆盖了 u 、 v 自己，以及 v 的所有其他子节点，覆盖范围完全包含 u ;

另一个常见的树上贪心策略：从叶子往上考虑。

为什么从叶子开始？

因为叶子只有一个邻居（它的父亲），自由度最低，最容易“被迫做出决定”。

这就是“叶子剥除”的思想——每次处理自由度最低的节点。

策略：

- 1 找到一个叶子 u ;
- 2 如果 u 还没被覆盖，选它的父亲 v 一定不劣于选 u 自己;
- 3 因为 v 覆盖了 u 、 v 自己，以及 v 的所有其他子节点，覆盖范围完全包含 u ;
- 4 删掉 v 及其所有子节点，继续向上。

时光倒流——逆向思维

核心思想：当问题的正向过程很复杂时，尝试从最终状态倒推。

时光倒流——逆向思维

核心思想：当问题的正向过程很复杂时，尝试从最终状态倒推。

POJ 3253 Fence Repair

一根长木板，每次切一刀分成两段，切割代价为当前木板长度，求切成长度分别为 A_i 的 N 段的最小代价。

正向思考很难：先切哪里？切的位置怎么选？

正向思考很难：先切哪里？切的位置怎么选？

但如果倒过来想：从 N 段小木板开始，每次选两段**合并**成一段，
代价为两段长度之和——这不就是**合并果子**吗？

正向思考很难：先切哪里？切的位置怎么选？

但如果倒过来想：从 N 段小木板开始，每次选两段**合并**成一段，
代价为两段长度之和——这不就是**合并果子**吗？

时光倒流把一个复杂问题转化成了我们已知的经典模型。

正向思考很难：先切哪里？切的位置怎么选？

但如果倒过来想：从 N 段小木板开始，每次选两段**合并**成一段，
代价为两段长度之和——这不就是**合并果子**吗？

时光倒流把一个复杂问题转化成了我们已知的经典模型。
这就是逆向思维的力量。

超级钢琴——前 K 大方案

P2048 NOI2010 超级钢琴

求序列中长度在 $[L, R]$ 内的所有子区间中，前 K 大的和。

区间总数是 $O(N^2)$ 级别，无法全部枚举。

区间总数是 $O(N^2)$ 级别，无法全部枚举。
但我们只需要前 K 大的， K 往往不大。

区间总数是 $O(N^2)$ 级别，无法全部枚举。

但我们只需要前 K 大的， K 往往不大。

核心思想：取出一个最优解后，通过分裂它的“决策区间”，无重复、无遗漏地生成后续次优解。

区间总数是 $O(N^2)$ 级别，无法全部枚举。

但我们只需要前 K 大的， K 往往不大。

核心思想：取出一个最优解后，通过分裂它的“决策区间”，无重复、无遗漏地生成后续次优解。

做法（堆 + 分裂）：

区间总数是 $O(N^2)$ 级别，无法全部枚举。

但我们只需要前 K 大的， K 往往不大。

核心思想：取出一个最优解后，通过分裂它的“决策区间”，无重复、无遗漏地生成后续次优解。

做法（堆 + 分裂）：

- 1 对每个左端点 i ，在其合法右端点区间 $[i + L - 1, i + R - 1]$ 内，用 RMQ（ST 表）找到使子段和最大的右端点 t ；

区间总数是 $O(N^2)$ 级别，无法全部枚举。

但我们只需要前 K 大的， K 往往不大。

核心思想：取出一个最优解后，通过分裂它的“决策区间”，无重复、无遗漏地生成后续次优解。

做法（堆 + 分裂）：

- 1 对每个左端点 i ，在其合法右端点区间 $[i + L - 1, i + R - 1]$ 内，用 RMQ（ST 表）找到使子段和最大的右端点 t ；
- 2 把 (i, t, l, r) 作为一个“候选方案”加入大根堆（按子段和排序）；

区间总数是 $O(N^2)$ 级别，无法全部枚举。

但我们只需要前 K 大的， K 往往不大。

核心思想：取出一个最优解后，通过分裂它的“决策区间”，无重复、无遗漏地生成后续次优解。

做法（堆 + 分裂）：

- 1 对每个左端点 i ，在其合法右端点区间 $[i + L - 1, i + R - 1]$ 内，用 RMQ（ST 表）找到使子段和最大的右端点 t ；
- 2 把 (i, t, l, r) 作为一个“候选方案”加入大根堆（按子段和排序）；
- 3 每次从堆顶取出最优方案，累加答案；

区间总数是 $O(N^2)$ 级别，无法全部枚举。

但我们只需要前 K 大的， K 往往不大。

核心思想：取出一个最优解后，通过分裂它的“决策区间”，无重复、无遗漏地生成后续次优解。

做法（堆 + 分裂）：

- 1 对每个左端点 i ，在其合法右端点区间 $[i + L - 1, i + R - 1]$ 内，用 RMQ（ST 表）找到使子段和最大的右端点 t ；
- 2 把 (i, t, l, r) 作为一个“候选方案”加入大根堆（按子段和排序）；
- 3 每次从堆顶取出最优方案，累加答案；
- 4 把右端点区间 $[l, r]$ 分裂成 $[l, t - 1]$ 和 $[t + 1, r]$ ，在两边继续找最优右端点，作为新方案入堆。

反悔贪心

事实上，贪心还有一部分极其重要的内容：反悔贪心。

反悔贪心

事实上，贪心还有一部分极其重要的内容：反悔贪心。
但是，由于它的难度较高，且模型较为灵活，展开讲要花费大量时间，因此受限于篇幅，我没有加入反悔贪心内容。

反悔贪心

事实上，贪心还有一部分极其重要的内容：反悔贪心。

但是，由于它的难度较高，且模型较为灵活，展开讲要花费大量时间，因此受限于篇幅，我没有加入反悔贪心内容。

学有余力的同学可以从我文末给出的题单中寻找反悔贪心的题目进行练习。一道非常经典的题目是 CF1974G，它同时也是西北多校算法竞赛联合训练 2026 年 6 月 21 日普及赛的 H 题。

进阶技巧小结

四个进阶技巧，本质是四种思维拓展。

进阶技巧小结

四个进阶技巧，本质是四种思维拓展。

- 树上合并：把复杂树形问题转化为“等效权值 + 并查集”；

进阶技巧小结

四个进阶技巧，本质是四种思维拓展。

- 树上合并：把复杂树形问题转化为“等效权值 + 并查集”；
- 叶子剥除：从自由度最低的节点开始，自底向上贪心；

进阶技巧小结

四个进阶技巧，本质是四种思维拓展。

- 树上合并：把复杂树形问题转化为“等效权值 + 并查集”；
- 叶子剥除：从自由度最低的节点开始，自底向上贪心；
- 时光倒流：正向复杂时，尝试从终点往回推；

进阶技巧小结

四个进阶技巧，本质是四种思维拓展。

- **树上合并**：把复杂树形问题转化为“等效权值 + 并查集”；
- **叶子剥除**：从自由度最低的节点开始，自底向上贪心；
- **时光倒流**：正向复杂时，尝试从终点往回推；
- **超级钢琴**：方案数爆炸时，用堆维护候选集，每次分裂产生新候选。

目录

- 1 引入
- 2 邻项交换
- 3 区间选点与区间覆盖
- 4 绝对值模型
- 5 哈夫曼模型
- 6 进阶技巧
- 7 总结

完整知识框架

部分	核心问题	工具
引入	怎么选最值?	sort + cmp
邻项交换	怎么定排序规则?	sort + 传递性
区间问题	怎么选点/选线段?	sort + 扫描
绝对值模型	怎么选位置?	sort + 中位数
哈夫曼模型	数据动态了怎么办?	动态数据结构
进阶技巧	难题怎么拓展?	并查集 / ST 表 / 堆

核心主线

整节课只有一个核心观点：

核心主线

整节课只有一个核心观点：

贪心决定“选什么”，数据结构决定“怎么快速选到”。

核心主线

整节课只有一个核心观点：

贪心决定“选什么”，数据结构决定“怎么快速选到”。

不要一上来就问“按什么排序”——那是跳过策略，直接问工具。

核心主线

整节课只有一个核心观点：

贪心决定“选什么”，数据结构决定“怎么快速选到”。

不要一上来就问“按什么排序”——那是跳过策略，直接问工具。先想清楚“选什么”，再想“怎么快速拿到”。无论题目怎么变，这两件事永远分开想：

核心主线

整节课只有一个核心观点：

贪心决定“选什么”，数据结构决定“怎么快速选到”。

不要一上来就问“按什么排序”——那是跳过策略，直接问工具。先想清楚“选什么”，再想“怎么快速拿到”。无论题目怎么变，这两件事永远分开想：

- 1 策略：**当前状态下，选哪个最优？

核心主线

整节课只有一个核心观点：

贪心决定“选什么”，数据结构决定“怎么快速选到”。

不要一上来就问“按什么排序”——那是跳过策略，直接问工具。先想清楚“选什么”，再想“怎么快速拿到”。无论题目怎么变，这两件事永远分开想：

- 1 策略：**当前状态下，选哪个最优？
- 2 工具：**怎么快速拿到它？——排序？堆？还是其他？

最后的话

贪心是一个“猜结论 + 证结论 + 实现”的过程。

最后的话

贪心是一个“猜结论 + 证结论 + 实现”的过程。
猜结论需要直觉，证结论需要逻辑，实现需要熟练使用工具。

最后的话

贪心是一个“猜结论 + 证结论 + 实现”的过程。
猜结论需要直觉，证结论需要逻辑，实现需要熟练使用工具。
三样东西，缺一不可。

最后的话

贪心是一个“猜结论 + 证结论 + 实现”的过程。
猜结论需要直觉，证结论需要逻辑，实现需要熟练使用工具。
三样东西，缺一不可。
希望这节课能帮你建立起一套**有章可循**的贪心思考框架。

课后思考题

难度较大，不懂的可以在群里提问/询问 AI。也可以看洛谷题解，但是洛谷题解质量不一定好，可能没有证明。

想不出来是正常的，尝试运用今天学到的算法过程即可，最后有没有推出来不要紧。前面这题现场可能也就一两支队伍通过了，后两题得出比较规则和严谨证明都非常困难。

P16444 XJTUPC 2026 I 题 Used to be （提示：考虑竖着看，每相邻两个字符串都可以有一个不同的位置，思考这个不同的位置如何利用）

P1080 NOIP 2012 提高组国王游戏 （提示：邻项交换推条件后得到一个 $\max(A, B) < \max(C, D)$ ，把 $\max$ 拆开可以得到四个有关 A, B, C, D 的条件，思考哪些一定成立，哪些一定不成立）

P2123 皇后游戏 （提示：直接邻项交换推出来的式子满足可传递性吗？会不会犯 CF2023A 一样的错误？）

推荐阅读

贪心题单: <https://www.luogu.com.cn/training/140652> (共包含三个难度)

超级钢琴技巧的深入学习:

<https://www.cnblogs.com/ljsj2009/p/18449654>

Thank you!